

# Software-Defined Perimeter (SDP) 규격서 v2.0



The permanent and official location for Software Defined Perimeter Working Group is  
<https://cloudsecurityalliance.org/research/working-groups/software-defined-perimeter-and-zero-trust/>

© 2022 Cloud Security Alliance – All Rights Reserved. You may download, store, display on your computer, view, print, and link to the Cloud Security Alliance at <https://cloudsecurityalliance.org> subject to the following: (a) the draft may be used solely for your personal, informational, non- commercial use; (b) the draft may not be modified or altered in any way; (c) the draft may not be redistributed; and (d) the trademark, copyright or other notices may not be removed. You may quote portions of the draft as permitted by the Fair Use provisions of the United States Copyright Act, provided that you attribute the portions to the Cloud Security Alliance.

# Acknowledgments

## Lead Authors

Jason Garbis  
Juanita Koilpillai

## Contributors

Junaid Islam  
Bob Flores  
Daniel Bailey  
Benfeng Chen  
Eitan Bremner  
Michael Roza  
Ahmed Refaey Hussein

## Special Thanks

Larry Hughes

## Reviewers

Alistair Cockeram Takahiro  
Ono  
T Prasad  
Nya Murray  
Michael Rash

## CSA Analyst

Shamun Mahmud

## CSA Global Staff

Claire Lehnert

## Version 1.0 Contributors

Brent Bilger, Alan Boehme, Bob Flores, Zvi Guterman, Mark Hoover, Michaela Iorga, Junaid Islam, Marc Kolenko, Juanita Koilpillai, Gabor Lengyel, Gram Ludlow, Ted Schroeder and Jeff Schweitzer

## Translators

박춘식

장현호

## CSA Analyst

The Software-Defined Perimeter (SDP) and Zero Trust Working Group is a Cloud Security Alliance (CSA) research working group that advocates for and promotes the adoption of Zero Trust security principles. It provides practical and sound guidance on how to apply Zero Trust to cloud and non-cloud environments. This group will build on and leverage the NIST Zero Trust research and methodologies. It will recommend SDP as a fundamental architecture for applying Zero Trust principles. It will revise and expand the SDP specification, to capture and codify a wealth of knowledge based on experience. Lastly, the SDP group recognizes the need for an inclusive approach to SDP. As such, it also supports alternative architectures so long as they align with Zero Trust principles.

# Dedication

*This paper is dedicated to the memory of Juanita Koilpillai, a security leader, influencer, mentor, and friend. Juanita was a luminary in her field and broke many glass ceilings on her path to becoming founder and chief executive at two successful start-ups. In return, Juanita shared her knowledge by mentoring young women with careers in technology. She played a prominent mentorship role in IEEE's Women In Engineering (IEEE WIE) Organization. She also created a philanthropic fund in memory of her father and founded MERGE, a non-profit to support local youth projects. She was instrumental in the authorship of this as well as several other CSA, IEEE and NIST publications.*

# 목차

|                                              |    |
|----------------------------------------------|----|
| Acknowledgments .....                        | 3  |
| Dedication .....                             | 4  |
| 목차.....                                      | 5  |
| 서론.....                                      | 8  |
| 목적.....                                      | 8  |
| 범위.....                                      | 8  |
| 예상 독자 .....                                  | 8  |
| SDP 설계.....                                  | 9  |
| SDP 개념.....                                  | 10 |
| SDP 아키텍처와 구성요소 .....                         | 11 |
| SDP 컨트롤러 .....                               | 13 |
| SDP Initiating Hosts (SDP 클라이언트).....        | 14 |
| SDP Accepting Hosts (AH).....                | 14 |
| SDP 배포 모델.....                               | 15 |
| SDP 워크 플로우.....                              | 17 |
| 컨트롤러 on-boarding 워크 플로우.....                 | 17 |
| Accepting Host (AH) Onboarding 워크 플로우 .....  | 18 |
| Initiating Host (IH) Onboarding 워크 플로우 ..... | 18 |
| 액세스 워크 플로우.....                              | 19 |
| IH On-boarding 워크 플로우 예 .....                | 20 |
| Single Packet Authorization (SPA).....       | 21 |
| SPA 메시지 포맷 .....                             | 22 |
| 안전한 자기 완결형 무연결형 메시지 전송 프로토콜, SPA .....       | 24 |
| SPA 대안 .....                                 | 24 |
| 컴포넌트 간 전송 계층 상호 인증.....                      | 25 |

|                                |    |
|--------------------------------|----|
| 디바이스 검증 .....                  | 25 |
| SDP 와 IoT 디바이스 .....           | 26 |
| 액세스 정책.....                    | 27 |
| SDP 프로토콜 .....                 | 29 |
| AH-컨트롤러 프로토콜 .....             | 30 |
| AH 와 컨트롤러 간 시퀀스 다이어그램 .....    | 30 |
| a. SPA.....                    | 30 |
| b. TCP 접속 및 상호 인증된 통신 설정 ..... | 31 |
| c. 로그인 (SDP join) 요청 메시지.....  | 31 |
| d. 로그인 (SDP Join) 응답 메시지.....  | 31 |
| e. 로그아웃 요청 메시지 .....           | 32 |
| f. Keep-Alive 메시지 .....        | 32 |
| g. AH 서비스 메시지.....             | 32 |
| h. 개인용 예약.....                 | 33 |
| IH-컨트롤러 프로토콜 .....             | 33 |
| IH 와 컨트롤러 간 시퀀스 다이어그램 .....    | 33 |
| a. SPA.....                    | 34 |
| b. TCP 접속 및 상호 인증된 통신 설정 ..... | 34 |
| c. 로그인 (SDP Join) 요청 메시지.....  | 34 |
| d. 로그인 응답 메시지.....             | 34 |
| e. Keep-Alive 메시지 .....        | 35 |
| f. IH 서비스 메시지 .....            | 35 |
| g. IH 인증 메시지 .....             | 36 |
| h. 로그아웃 요청 메시지 .....           | 37 |
| z. 개인용 예약.....                 | 37 |
| IH-AH 프로토콜.....                | 37 |
| IH 와 AH 간 시퀀스 다이어그램 .....      | 37 |
| a. SPA.....                    | 38 |
| b. TCP 접속 및 상호 인증된 통신 설정 ..... | 38 |

|                               |    |
|-------------------------------|----|
| c. TCP 접속 요청 메시지 .....        | 39 |
| d. 적절한 접속 유형 설정 .....         | 39 |
| e. TCP 접속 응답 메시지 .....        | 39 |
| f. 데이터 메시지 .....              | 39 |
| g. 접속 종료 메시지 .....            | 40 |
| z. 사용자 정의 메시지 .....           | 40 |
| 로깅 .....                      | 40 |
| 로그 메시지의 필드 .....              | 40 |
| 운영 로그 .....                   | 40 |
| 보안 / 접속 로그 .....              | 41 |
| 요약 .....                      | 43 |
| References .....              | 44 |
| 부록 A: SDP, SDN & NFV .....    | 46 |
| 부록 B: OSI / SDP 컴포넌트 매핑 ..... | 47 |

# 서론

SDP(Software-Defined Perimeter) 아키텍처는 네트워크 보안 경계를 동적으로 유연하게 배포하여, 안전이 보장되어 있지 않은 네트워크에 있는 애플리케이션이나 서비스를 격리하는 기능을 제공한다. SDP는 격리에 필요한 온디맨드(on-demand)로, 동적으로 프로비저닝 되는(provisioned) 신뢰의 오버레이(overlay)를 제공하도록 설계되어 기업이 관리하는 네트워크 내부 및 외부로부터의 네트워크 기반의 공격을 경감하도록 설계되어 있다. SDP 구현은 승인되지 않은 엔티티(unauthorized entities)로부터 자산을 은폐하고, 접속을 허가하기 전에 신뢰를 확립하여, 분리된 제어 플레인(control plane)과 데이터 플레인(data plane)에 의해서 시스템을 관리한다. SDP의 도입 시 기업은 제로 트러스트의 목표를 달성하는 것이 가능하며, 그 결과, 종래의(대체로 비효율적인) 경계 중심 모델로부터 탈각하여, 보안의 유효성과 회복력을 향상시키는 것이 가능하다.

## 목적

본 규격은, 2014년 4월에 SDP WG(Software Defined Perimeter Working Group)이 공개한 CSA(Cloud Security Alliance) SDP(Software-Defined Perimeter) Version 1.0을 경신한 것이다.

기존의 규격도 안정된 것이지만, 컴포넌트의 온보딩(onboarding)이나 non-person entities<sup>1</sup>의 보호 등 몇 가지 토픽(topic)에 충분히 대응하고 있지 못하였다. 또한 SDP v1 출시 후, 업계는 SDP의 원칙을 적극적으로 받아 들여, 현재 제로 트러스트에 포함하였다. 이 개정판에는, 보다 많은 토픽에 대한 대응이나 기술 내용의 개선에 더불어, 제로 트러스트를 둘러싼 업계의 현황을 반영하였다.

이 개정판은, SDP WG이 SDP v1 이후에 공개한 문서, 특히 SDP 용어집과 SDP 아키텍처 가이드를 기반으로 한다. 이 2개의 문서에 대한 링크는, 참고 문헌 세션에 기재되어 있다.

## 범위

본 규격은 SDP에 있어서 아키텍처의 구성 요소, 상호 작용 및 기본적인 보안 통신 프로토콜을 다루고 있다. 보안 경계 내 안전한 접속을 가능하게 하는 제어 플레인과 서버, 디바이스, 서비스 등의 시작 호스트(initiating host)와 수락 호스트(accepting host) 사이의 안전한 접속을 실현하는 데이터 플레인에 초점을 맞추고 있다.

## 예상 독자

본 규격의 대상 독자는 다음과 같다.

- 제로 트러스트 및 SDP 제품을 기업에서 도입하고 있는 솔루션 아키텍처 및 보안 리더
- SDP 아키텍처를 사용하여 제로 트러스트를 제품에 실장하고 있는 벤더나 기술 제공 업체

<sup>1</sup> non-person entities (NPE)는, 인간은 아니지만 전자 신원을 가지고, 사이버 공간에서 활동하는 실체이다. 하드웨어 장치, 소프트웨어 애플리케이션, 및 정보 성과물 등이 포함된다.



# SDP 설계

SDP는 기업 보안 아키텍처, 네트워크 제공자, 어플리케이션 소유자에게, 다음과 같은 기능을 제공하는 것을 목적으로 하고 있다.

- 동적 「소프트웨어 정의」 경계 배치
- 네트워크와 리소스 은폐
- SDP 상에서 동작하는 서비스에 대한 승인되지 않은 액세스 방지
- 신원(Identity) 중심 한 액세스 정책 모델 적용

SDP는 물리적인 어플라이언스를 물리적으로 배치된 장소에 관계없이, 조직의 관리 하에서 동작하는 논리적 컴포넌트로 대체함으로써 논리적인 경계를 최소화한다. SDP는 최소 권한에 의한 액세스, 침해의 가정(assuming breach), 「신뢰하지만 검증한다」라고 하는 제로 트러스트의 원칙을 적용하여, 인증과 신원 확인이 된 경우에 한하여 정책 기반 리소스 액세스를 허가한다.

SDP의 설계 목적은 IPv4 및 IPv6 네트워크에 효과적이면서 용이하게 통합할 수 있는 보안 아키텍처를 제공하는 것이다. 설계 목적에는, 제어 플레인 요소의 은폐 및 액세스 제어를 포함한다.

SDP는 데이터 플레인에 흐르는 통신에 기밀성과 무결성을 제공한다. 또한 경계에 암호 기술을 사용하여 사인인(sign in) 하기 위해 검증된 디바이스 상에서 신원(사용자 또는 non-person entity)의 인증을 행하여 Need-to-know의 원칙에 따른 액세스 모델에 대응한다.

SDP의 설계는 사용자, 사용자의 디바이스, 네트워크, 디바이스에 대한 보안을 통합하는 여러 계층에서 단절 없이(seamless) 통합을 제공한다. SDP는 기존 하드웨어 기반, SDN(Software Defined Network), 클라우드 기반에 관계없이, 모든 IP 인프라에서 접속을 보호하는 데 사용된다.

SDP의 상호 인증 터널은 효과적으로 암호화된 오버레이(overlay)로, 모든 종류의 IP 네트워크 상에 배치할 수 있다. 그 결과 SDP는 다른 환경 전반에서 보안 계층을 정규화하여 네트워크, 보안 및 운영을 간소화한다.

클라우드 인프라의 경우, SDP는 OSI의 7계층 가운데 다음의 5계층에서 보안을 통합한다:

- 네트워크 계층 : 가상화<sup>2</sup>에 의해서, 컴퓨팅, 스토리지, 모니터링을 제공한다.
- 전송 계층 : 클라우드 API는 가상화된 자산을 리소스 풀 및 사용자와 연결한다.
- 세션 계층 : 아래 층에 있는 가상화된 인프라를 관리한다.
- 표현 계층 : 미들웨어는 응용 계층과 애플리케이션을 관리한다.
- 응용 계층 : 사용자에게 비즈니스 가치를 제공한다.

<sup>2</sup> SDP 및 NFV(Network Function Virtualization)에 관한 논문은 다음과 같다 - <https://www.waverleylabs.com/resources/publications/>  
© Copyright 2022, Cloud Security Alliance. All rights reserved.

SDP가 통합되지 않는 OSI 계층은 데이터 링크 계층과 물리 계층이다. TCP/IP 모델은 이러한 결합된 계층을 네트워크 계층이라고 한다.

SDP 및 네트워크 계층 모델에 대한 자세한 내용은 부록 B를 참조하기 바란다.

SDN 및 NFV(Network Function Virtualization)를 보완하는 것으로, SDP는 SDN이 구축하는 IP 인프라를 통해 접속을 보호할 수 있다.

이 경신판의 규격에는, 지금까지의 피드백과 SDP의 구현으로부터 얻은 교훈을 기반으로, 초판의 규격에서 정의되고, SDP 아키텍처 가이드<sup>3</sup>에서 상세히 설명한 다양한 배치 모델을 소재로, 액세스 제어의 과제를 명확히 하는 데 도움이 된다.

SDP는 어플리케이션 및 인프라에 대한 네트워크 공격이나 cross-domain 공격을 방지, 탐지, 및 대응에 있어서 대단히 뛰어난 접근 방식을 제공한다. 사용자가 인터넷과 같은 신뢰되지 않는 공용 네트워크를 경유하여 리소스에 액세스하는 경우에도 공격 대상면을 최소화하고, 최소 권한의 원칙을 적용함으로써 실현된다.

기존의 사이버 보안 솔루션은 네트워크나 시스템 보호에 중점을 두는 반면, SDP는 Identity 중심의 관점에서 자산을 보호하는 것에 중점을 두고 있다. 경계 중심의 보안에서부터 이행함으로써 기업은 기업이 관리하는 private 리소스에 대한 DDoS, 자격 증명 탈취, 랜섬웨어 등의 공격에 대한 내성을 높일 수 있다.

## SDP 개념

SDP는 조직의 경계가 아니라, 조직의 중요한 리소스를 보호하는 것에 중점을 두고 있다. 리스크기반으로, 동적, Identity 중심 및 상황 별 액세스 정책을 네트워크의 모든 계층에 정의하여 적용할 수 있다. SDP는 조직 내에서 처음으로 비즈니스, 애플리케이션, 네트워크의 관계자에 의해서 의미 있는 액세스 정책을 정의하고, 적용할 수 있는 기반을 제공한다.

SDP는 애플리케이션이나 기업 리소스 소유자에게, 경계를 배치하는 능력을 제공한다:

- 침해되어 있을 지도 모르는 네트워크에 안전하게 서비스를 배포한다(즉, 「침해를 가정」한다).
- 신뢰되지 않는 네트워크를 경유한 서비스에 대해 세부 액세스 권한을 가진 Identity를 제공한다. 이러한 유스케이스의 하나로서, VPN 교체가 있다.

<sup>3</sup> <https://cloudsecurityalliance.org/artifacts/sdp-architecture-guide-v2/>

SDP 는 경계 기반의(대계, 물리적인) 어플라이언스를 애플리케이션 소유자의 관리 하에서 동작하는 논리적인 컴포넌트로 대체한다. SDP 는 액세스 정책에 따라서, 디바이스의 인증과 Identity 의 검증을 행한 후에 한하여 애플리케이션에 대한 액세스를 제공한다.

SDP 의 배후에 있는 원리는 완전하게 새로운 것은 아니다. 미 국방부 및 정보 기관(Intelligence Community, IC) 내 다수 조직에서 네트워크 액세스 전의 인증 및 인가를 기반으로 하여 이와 같은 네트워크 아키텍처를 구현했다. 일반적으로 미 국방부가 정의한 기밀성이 높은, 「high-side」 네트워크에서 사용되는 모든 서버는 원격 액세스 게이트웨이 어플라이언스의 배후에 은폐되어, 사용자는 인가된 서비스를 확인하고 액세스를 허가하기 전에 어플라이언스에 대해서 인증을 해야 한다. SDP 는 계층화된 네트워크에서 사용되고, 표준 워크플로우에 통합된 논리적 모델을 활용했다. 보안 리더들은 2004 년 Jericho Forum 포럼을 시작으로 오랜 기간에 걸쳐서 이러한 개념들 중 대부분을 지지해 왔다. 최근 미국 국립표준기술연구소(NIST)가 정의한 「제로 트러스트」에도 이러한 원칙을 통합하고 있다.<sup>4</sup>

SDP 는 위에서 설명한 need-to-know(최소 권한) 모델의 이점을 유지하면서, 원격 액세스 게이트웨이 어플라이언스를 필요로 하는 단점을 배제하고 있다. 실제 SDP 는 「원격」 사용자만이 아니라, 「모든」 사용자에게 액세스 제어를 적용하도록 설계되었다. SDP 는 보호된 서버나 서비스에 대한 네트워크 액세스를 얻기 전에 End Point 의 인증과 인가를 요구한다. 그 후, 액세스의 시스템과 애플리케이션 인프라와의 사이에 암호화된 접속을 실시간으로 생성한다. 즉, SDP 는 사용자, 디바이스, 서비스 등의 리소스를 경계에서 안전하게 인증하고, 미인가의 리소스에는 서비스 은폐된 상태로 남아있다.

## SDP 아키텍처와 구성요소

가장 단순한 경우 SDP 는 SDP 호스트(SDP Host)와 SDP 컨트롤러(SDP Controller)와 같은 2 가지 논리 컴포넌트로 구성된다. full stack 호스트 또는 경량 서비스와 같은 SDP 호스트는 접속의 개시 또는 허용할 수 있다. 이러한 작업은 「제어 플레인(Control Plane)」의 안전한 채널을 통하여 SDP 컨트롤러와의 상호작용하여 관리된다. 데이터는 독립된 「데이터 플레인(Data Plane)」의 안전한 채널을 통해서 통신된다. 제어 플레인이 데이터 플레인으로부터 분리되어 있기 때문에, 유연하며 확장성이 높은 아키텍처의 시스템을 실현하는 것이 가능하다. 또한 확장성이나 가용성을 위해, 모든 컴포넌트를 다중화 할 수 있다. SDP 호스트(Initiating 또는 Accepting)는 어플라이언스 또는 서버 프로세스인 SDP 컨트롤러와 협조하여 사용자가 인증 · 인가되고, 디바이스가 검증되고, 안전한 통신이 확립되어, 네트워크 상의 데이터와 제어용의 트래픽이 분리되는 것을 확실하게 한다.

<sup>4</sup> NIST Zero Trust Architecture - SP 800-207 <https://csrc.nist.gov/publications/detail/sp/800-207/final>  
© Copyright 2022, Cloud Security Alliance. All rights reserved.

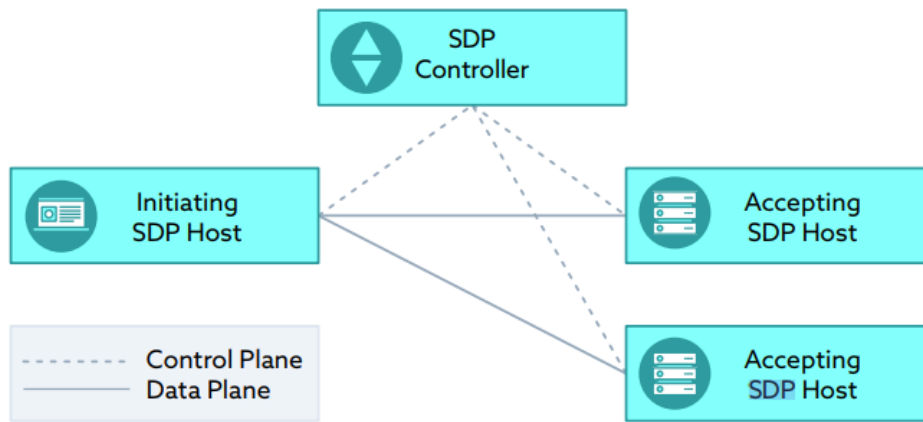


Figure 1: SDP 아키텍처 (이전 CSA 에서 공개한 Software Defined Perimeter 와 Zero Trust)<sup>5</sup>

SDP 아키텍처는 다음과 같은 컴포넌트로 구성된다.

- **SDP 컨트롤러** – 이 컴포넌트는 모든 인증과 액세스 플로우를 관리하도록 설계되어 있다. 액세스 정책을 정의하고, 평가하기 위한, 솔루션의 본질적인 「두뇌」이다. 제로 트러스트 정책 결정 포인트(PDP) 역할을 한다. SDP 컨트롤러는 기업의 인증 솔루션(Identity 제공자, 다중 요소 인증 서비스 등)과 연계하여 인증과 인가의 오케스트레이션을 담당한다. 정의된 액세스 정책에 의해서 허가된 접속을 가시화하고, 보고하기 위한 제어 포인트 역할도 한다.
- **Initiating Hosts (IH)** – 이러한 액세스 Entity 는 사용자의 디바이스, 하드웨어(End User 디바이스 또는 서버 등), 네트워크 디바이스, 소프트웨어 애플리케이션, 서비스 등의 NPE(No Person Entity)이다. SDP 사용자는 SDP 클라이언트 또는 브라우저를 사용하여 통신을 개시할 수 있다
- **Accepting Hosts (AH)** - 이러한 Entity 는 SDP 에 의해서 보호 및 액세스되는 애플리케이션이나, 서비스, 리소스의 앞에 존재하는 논리적 컴포넌트이다. AH 는 제로 트러스트 정책 시행 지점(Policy Enforcement Point, PEP)로서 기능한다. PEP 는 SDP 전용의 소프트웨어 또는 하드웨어로서 구현될 수 있다. AH 는 SDP 컨트롤러로부터의 지시에 따라 대상 서비스 (애플리케이션, 경량 서비스 또는 리소스)에 대한 네트워크 트래픽을 허가 또는 거부한다. 논리적으로 AH 는 대상 서비스와 공존하거나 네트워크로 분리될 수 있다.

이러한 SDP 컴포넌트는 온프레미스 또는 클라우드에 배치하는 것이 가능하며, 확장성이나 가용성을 위해 다중화하여 배치할 수 있다.

여기서부터, 각 컴포넌트에 대해 살펴보도록 한다.

<sup>5</sup> <https://cloudsecurityalliance.org/artifacts/software-defined-perimeter-and-zero-trust/>

## SDP 컨트롤러

SDP 컨트롤러는 정책을 정의, 검증 및 결정하는 메커니즘으로, 제로 트러스트의 정책 결정 포인트로서, 어느 Identity (사용자나 그룹 등)가 어떠한 디바이스로부터 조직의 서비스(온 프레미스 또는 클라우드)에 액세스할 수 있는가에 대한 정보를 관리한다. 그리고 어느 SDP 호스트가 상호 통신할 수 있는지를 결정한다.

IH 상의 사용자가 컨트롤러에 접속하면, 컨트롤러는 사용자를 인증하여 Identity 나 디바이스 속성을 포함하는 사용자의 컨텍스트에 따라 사용자에게 허가된 서비스(AH 를 통하여)에 한하여 액세스 권한을 부여한다.

사용자를 인증하기 위해, 컨트롤러는 내부 사용자 테이블을 사용하거나, 3rd party 의 IAM(Identity and Access Management) 서비스(온 프레미스 또는 클라우드)에 접속하거나, 다중 요소 인증(MFA)을 적용할 수 있다. 일반적으로 인증은 사용자 유형 및 Identity 를 기반으로 처리된다. 예를 들면, 직원은 Identity 제공자를 통해서 인증되는 한편, 계약 직원은 데이터베이스에 저장된 인증 정보나 Identity Federation 에 의해서 인증될 수 있다.

사용자에게 서비스에 대한 액세스를 허가하기 위해, 컨트롤러는 내부의 사용자 · 서비스 매핑 및 정책 모델 또는 LDAP, Active Directory, 기타 권한 부여 솔루션 등 3rd party 의 서비스(온 프레미스 또는 클라우드)를 사용하는 것이 가능하다. 일반적으로 인가는 사용자의 역할과 사용자나 디바이스의 속성 또는 사용자가 액세스할 수 있는 실제 데이터 요소나 데이터 플로우에 기반한 상세한 정보에 의해서 결정된다. 실제 SDP 컨트롤러가 유지하는 액세스 정책은 조직 서비스 디렉토리나 Identity 스토어 등의 다른 조직 구성 요소를 통해 알릴 수 있다. 이와 같이 컨트롤러는 NIST 가 제로 트러스트의 이념으로 내세우는 동적 제로 트러스트 정책 유형을 실현한다.

더욱이 컨트롤러는 지리적 정보나 호스트 유효성 검증 서비스 등 외부 서비스로부터 정보를 취득하여 IH 상의 사용자의 유효성을 추가로 검증할 수 있다. 또한, 컨트롤러는 사용자의 인증 실패 또는 기밀성이 높은 서비스에 대한 액세스와 관련된 상황 별 정보를 다른 네트워크 컴포넌트에 컨텍스트 정보를 제공할 수 있다.

제로 트러스트 PDP 에 대응하는 SDP 컨트롤러는 SDP 아키텍처에 따라 클라우드 또는 온 프레미스에 배치할 수 있다.

SDP 컨트롤러는 SPA(Single Packet Authorization) 프로토콜을 사용하는 격리 메커니즘에 의해 보호되고 있어 미승인 사용자나 디바이스로부터 은폐되어 액세스할 수 없다. 이 메커니즘은 컨트롤러 앞에 SDP 게이트웨이를 설치하여 제공되거나 컨트롤러 자체 내에 기본적으로 제공될 수 있다.

## SDP Initiating Hosts (SDP 클라이언트)

SDP IH 는 호스트 승인을 통해 외부로부터 보호된 회사 리소스에 액세스하는 프로세스를 시작하기 위해 SDP 컨트롤러와 통신한다.

컨트롤러는 인증 단계에서 IH 가 사용자 ID, 하드웨어 또는 소프트웨어 인벤토리, 디바이스 상태 등의 정보를 제공해야 한다. 또한, 컨트롤러는 IH 가 AH 와의 보안 통신을 설정하기 위한 메커니즘(예: 자격 증명)을 제공해야 한다.

IH 는 최종 사용자(end-user) 컴퓨터 또는 웹 브라우저에 설치된 클라이언트 응용 프로그램 형태일 수 있다. 클라이언트 응용 프로그램을 사용할 경우, 호스트 확인(디바이스 상태 확인), 트래픽 라우팅 및 간소화된 인증 등 많은 기능을 제공한다.

IH(Initiating Host)의 가장 중요한 역할은 문서 뒷부분에 설명한 SPA 를 사용하여 접속을 시작하는 것이다. SPA 패킷은 구현 시 브라우저 기반 SDP 클라이언트에 의해 생성될 수 있다. IH 는 사용자의 디바이스(예: 직원 또는 외부 인력 컴퓨터 또는 모바일 디바이스), 애플리케이션(예: 팟 클라이언트(다양한 기능을 제공하는 클라이언트)) 또는 사물 인터넷(IoT) 디바이스(예: 원격 양수기)일 수 있다. 위 예시 중 후자의 경우, ID 는 인증되고 허가되었지만 결과적으로는 비인간(non-human) ID 이다. 자세한 내용은 「SDP 및 IoT 디바이스」 절에서 참고 바란다.

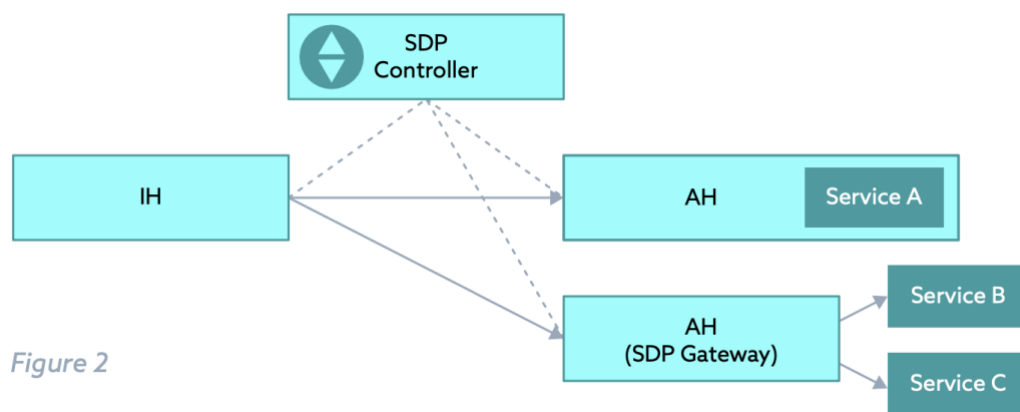
## SDP Accepting Hosts (AH)

Accepting Host(AH)는 필요한 ID 에 접속을 허용하는 한편, 기업이 책임을 가지고 액세스를 은폐 및 제어하는 등 SDP 의 정책 시행 지점 (PEP)로서, 리소스(또는 서비스)에 대한 액세스를 제어한다. AH 는 온 프레미스, Private cloud, Public cloud 등에 배치할 수 있다.

AH 로 보호되는 서비스는 웹 애플리케이션에 한정되지 않는다. 예로, 팟 클라이언트가 액세스하는 TCP 나 UDP 기반의 어플리케이션(SSH, RDP, SFTP, SMB 등) 또는 독점 애플리케이션이 될 수 있다.

AH 에 대한 모든 네트워크 액세스는 디폴트로 차단되며, 인증 및 허가 받은 entity 에 한하여 액세스할 수 있다.

앞서 설명한 바와 같이, AH 는 대상 서비스와 공존하거나 네트워크를 통해 분리될 수 있다. 이러한 모델은 아래 그림 2 와 같다.



AH 는 SDP 컨트롤러로부터 제어 정보를 수신하여 컨트롤러의 지시를 받은 경우에 한하여, IH 로부터의 접속을 승인한다. AH 는 SDP 컨트롤러로부터 수신한 제어 정보를 이용하여 인가된 IH(사용자 및 디바이스)에 대해서 보호된 서비스에 대한 액세스를 제공한다.

AH 는 IH 로부터 트래픽을 수신하여 보호된 백엔드 서비스로 중계하기 위해, 암호화 통신 또는 액세스를 위한 교환기 역할을 한다. 이후 백엔드 서비스의 응답은 IH 로 다시 전송된다.

SDP 는 논리적 연결 지향 프로토콜로, 다양한 네트워크 배포 토폴로지에 대응한다. 그림 3 에서 소개하는 다양한 배포 모델 및 아키텍처는 SDP 아키텍처 가이드에 상세하게 설명되어 있다. 다양한 배포 모델에 맞는 SDP 컴포넌트의 구성을 나타내고 있다.

## SDP 배포 모델

SDP 는 다음에서 설명하는 다이어그램에서 서버로 표시된 리소스에 클라이언트(인간 또는 Non Person Entity)를 연결한다. 이러한 리소스는 네트워크를 통해 통신이 될 경우 제약이 없다. 물리적 또는 가상 서버에서 실행되는 서비스, IaaS, PaaS 플랫폼에서 실행되는 서비스 또는 컨테이너형 서비스 등 일 수 있다.

이 절에서는, 간략하게 6 가지 SDP 구축 모델을 소개한다. 서로 다른 네트워크 토폴로지를 사용하지만 보호된 리소스에 대한 액세스를 강제하는 것과 동일한 이점을 논리적으로 제공한다. 이러한 배포 모델에 대한 상세한 소개는 「SDP Architecture Guide」<sup>6</sup>를 참고하기 바란다.

아래 다이어그램에서 파란색 선은 MITM 공격에 대한 대책으로, mTLS 또는 Internet Key Exchange(IKE) 등의 상호 인증된 암호 프로토콜에 의해서 보호된 네트워크 접속을 나타낸다. 회색 선은 암호화되거나 암호화되지 않을 경우 Native 애플리케이션 프로토콜을 나타낸다. 이 다이어그램에서는 간략화하기 위해, SDP 컨트롤러를 생략한다.

일부 배포 모델은 SDP 게이트웨이를 사용한다. SDP 게이트웨이는 SDP Accepting Host 로서 동작하는 소프트웨어 에이전트이며, 보호 대상 서비스의 격리와 액세스 제어를 제공한다.

<sup>6</sup> For a complete description, see the CSA Software-Defined Perimeter Architecture Guide, May 2019, pages 14 to 18  
© Copyright 2022, Cloud Security Alliance. All rights reserved.

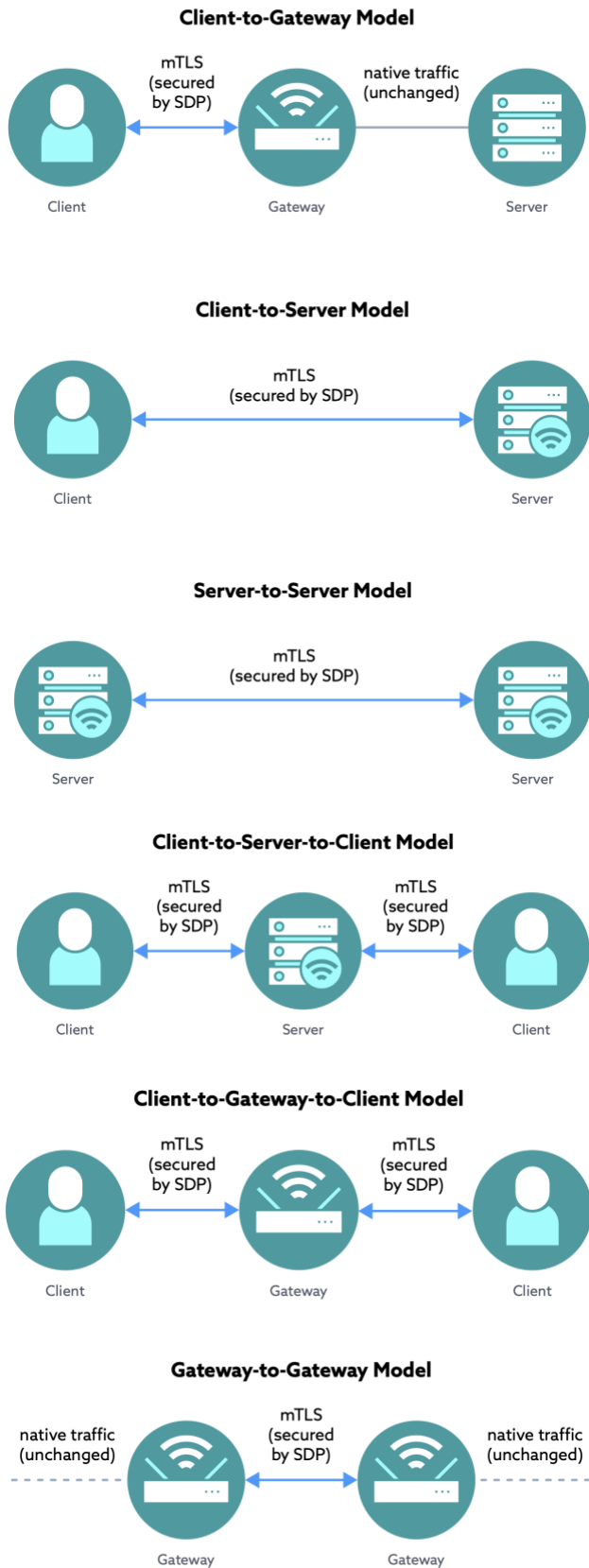


Figure 3 - SDP Deployment Models

SDP 게이트웨이에서 하나 이상의 서버를 보호해야 하는 경우 기본 네트워크 토폴로지에 관계없이 IH 와 AH(게이트웨이) 간의 연결이 보호된다. Client-to-Gateway 모델에서 게이트웨이는 원격으로 액세스할 수 있지만 은폐되어 있으며 안전한 경계를 제공한다. 이 모델에서는 보호 대상의 서버를 변경할 필요가 없다.

조직에서 End To End 통신을 보호해야 하는 경우, Client-to-Server 모델은 서비스와 AH 를 단일 호스트로 통합한다. 이 모델에서 서버는 보안 경계 내 은폐된다. 이 모델에서 서버가 AH 로 동작하기 위해서는 SDP 소프트웨어가 있어야 한다.

Server-to-Server 모델은 계층이 낮은 네트워크에 관계없이, 서버 사이의 모든 접속이 암호화되어 있는 것을 보증한다. 이 모델에서는, 모든 서버가 안전한 경계 내 은폐된다.

VOIP, 채팅 및 화상 회의 서비스와 같은 peer-to-peer 트래픽이 중계 서버를 경유하는 경우가 있다. Client-to-Server-to-Client 모델에서 서버는 보안 경계 내 은폐된다.

Client-to-Gateway-to-Client 모델은 Client-to-Server-to-Client 모델의 변형이다. 이 모델은 액세스 정책을 적용하면서, 클라이언트가 상호 직접 접속하는 것을 필요로 하는 peer-to-peer 의 네트워크 프로토콜을 지원한다. 이 모델에서 게이트웨이는 경계 내 은폐된다.

Gateway-to-Gateway 모델은 SDP Specification 1.0 에 포함되어 있지 않다. 이 모델은 특정 IoT 환경에 적합하다. 이 모델에서 게이트웨이는 경계 내 은폐된다.



# SDP 워크 플로우

일반적으로, SDP 컴포넌트의 워크 플로우는 on-boarding 워크 플로우(각 컴포넌트에 개별 플로우)와 액세스 워크 플로우(컴포넌트 간 조정된) 2 종류로 나뉜다. 정의에 의하면, 각 SDP 컴포넌트에는 하나의 on-boarding 워크 플로우가 있으며, 이후 여러 액세스 워크플로우에 참가한다.

실 구축 또는 SDP 배포 모델에 따라 구체적인 내용이 상이하므로 이러한 워크 플로우가 대표적이다.

## 컨트롤러 on-boarding 워크 플로우

SDP 시스템에는 하나 이상의 컨트롤러가 필요하다. On-boarding 워크 플로우가 성공하기 위해서는, 적어도 하나의 컨트롤러가 항상 사용 가능해야 한다. 일부 실 구축 시 액세스 워크 플로우를 성공시키기 위해, active 컨트롤러가 필요한 경우도 있다. 컨트롤러는 다른 SDP 컴포넌트가 작동하는 모든 위치에서 네트워크에 액세스할 수 있어야 한다. 따라서 인증된 사용자나 디바이스에 한하여 인터넷에서 글로벌하게 액세스할 수 있다.

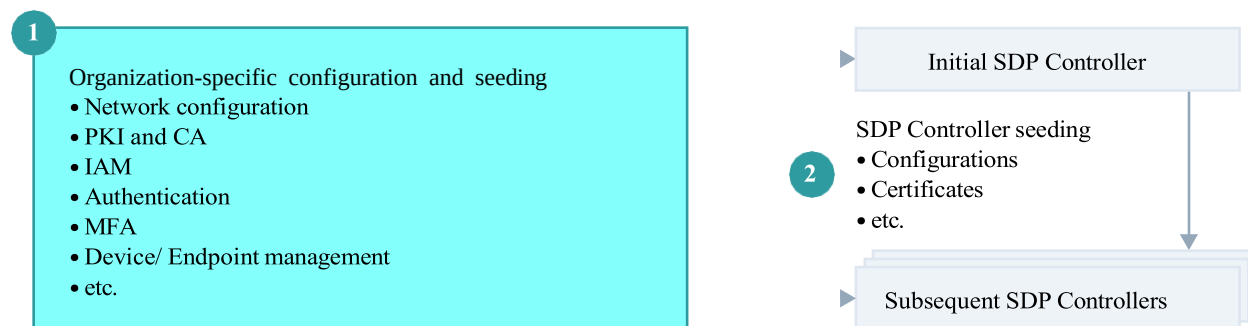


Figure 4 - Controller Onboarding Workflow

그림 4 는 컨트롤러 on-boarding 워크 플로우를 나타낸다. 초기(primary) SDP 컨트롤러가 기동하여, 적절한 인증 및 권한 부여 서비스 (예: PKI 발급 인증 기관 서비스, 장치 증명, Geolocation, SAML, OpenID, OAuth, LDAP, Kerberos, 다중 요소 인증 등)와 접속된다. SDP 컨트롤러는 다른 SDP 컴포넌트가 즉시 사용할 수 있도록 무제한으로 가동된다. 그 다음의 컨트롤러를 온라인으로 전환하여 조직 고유의 동일한 구성을 조합하고 초기 컨트롤러로부터 시드 정보의 인계로 on-board 하는 것도 가능하다.

대부분의 SDP 구현은 로드 밸런싱 및 이중화를 목적으로 컨트롤러 클러스터 구성을 지원한다. 각 SDP 구현은 그 다음의 컨트롤러를 클러스터 내 다른 컨트롤러에 접속하여, 적절한 상태를 공유하는 방법을 지원해야 한다. 이 메커니즘은 구현에 따라 다르며, 본 규격의 범위에서 벗어난다.

## Accepting Host (AH) Onboarding 워크 플로우

모든 SDP 시스템은 1 개 이상의 AH 가 필요하다. AH 는 앞서 설명한 SDP 배치 모델을 통해 구현할 수 있다. 즉, standalone 게이트웨이를 사용하거나, 리소스 또는 워크로드를 제공하는 서버의 일부로 배포될 수 있다.

SDP 구현 시, AH 는 장기간 가동되거나 일시적일 수 있다. Standalone 게이트웨이 AH 는 수 개월 또는 수 년 등 장기간 가동될 수 있다. 하지만, 부하에 따라 확장 또는 축소되는 게이트웨이의 동적 클러스터 내에서는 단기간 가동되는 경우도 있다.

개별 서버 내 배치된 AH 의 경우, 마찬가지로 장기간 또는 단기간 가동될 수 있다. 이 경우, AH 가동 시간은 AH 가 속한 서버 인스턴스의 가동 시간과 연동한다. 서버 인스턴스는 기존 웹 서버나 애플리케이션 서버와 같이 장기간 가동되거나 DevOps 인프라의 일부와 같이 단기간이 될 수도 있다.

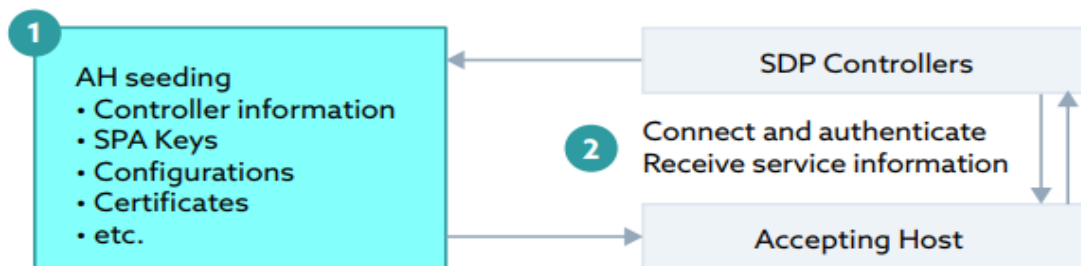


Figure 5 - Accepting Host Onboarding Workflow

그림 5 는 AH 온보딩 워크 플로우를 나타낸다. AH 가 서비스를 시작할 때 SDP 에 있는 하나 이상의 컨트롤러에 접속하여 인증해야 한다. AH 가 on-board 될 경우, SPA 패킷을 수신하여 인가된 IH 로부터의 액세스를 처리할 준비가 된다.

각 SDP 구현은 AH 가 해당 클러스터의 컨트롤러에 연결하도록 구성된 매커니즘을 지원해야 한다. 이 매커니즘은 구현에 따라 상이하며, 이 규격의 범위를 벗어난다. 마찬가지로, 대부분의 SDP 구현에서는 로드밸런스와 이중화를 목적으로 AH 클러스터의 구현 별 배포를 지원하는데 가장 일반적으로는 게이트웨이 구축 모델 중 하나를 사용한다.

## Initiating Host (IH) Onboarding 워크 플로우

IH(Initiating Hosts)는 사용자 디바이스, IoT 디바이스 또는 IH 역할을 하는 서버와 같이 사용자가 없는 시스템일 수 있다. SDP 시스템의 모든 구성요소와 마찬가지로, IH 는 on-board 가 필요하다(그림 6 참조). 이러한 워크플로우가 진행되는 동안 컨트롤러에 연결하는 데 필요한 초기 정보로 구성된다. 여기에는 네트워크 정보(호스트 명, IP 주소)와 필요한 공유 secret(SPA 키, 증명서 등)이 포함된다. 이 매커니즘은 구현에 따라 상이하며, 이 규격의 범위를 벗어난다. 이 경우 일반적으로 엔터프라이즈 ID 관리 제공자를 사용할 필요가 있으며, 사용자 디바이스의 경우 사용자 인증은 IH on-boarding 워크 플로우 내 단계인 경우가 많다.

IH 는 한 번 on-boarding 할 필요가 있어, 이후 액세스 워크 플로우를 시작할 수 있다.

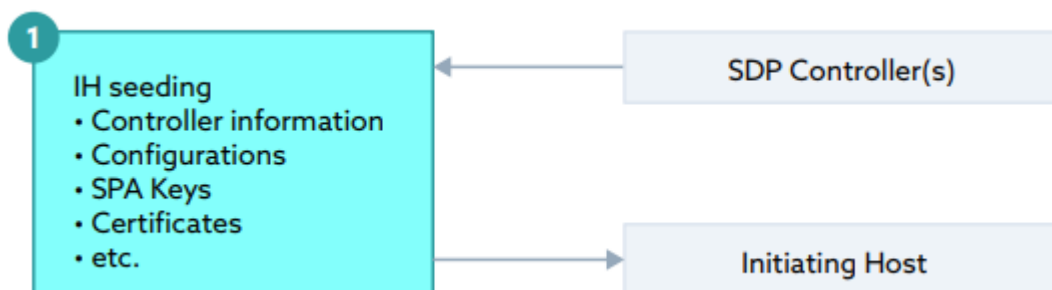


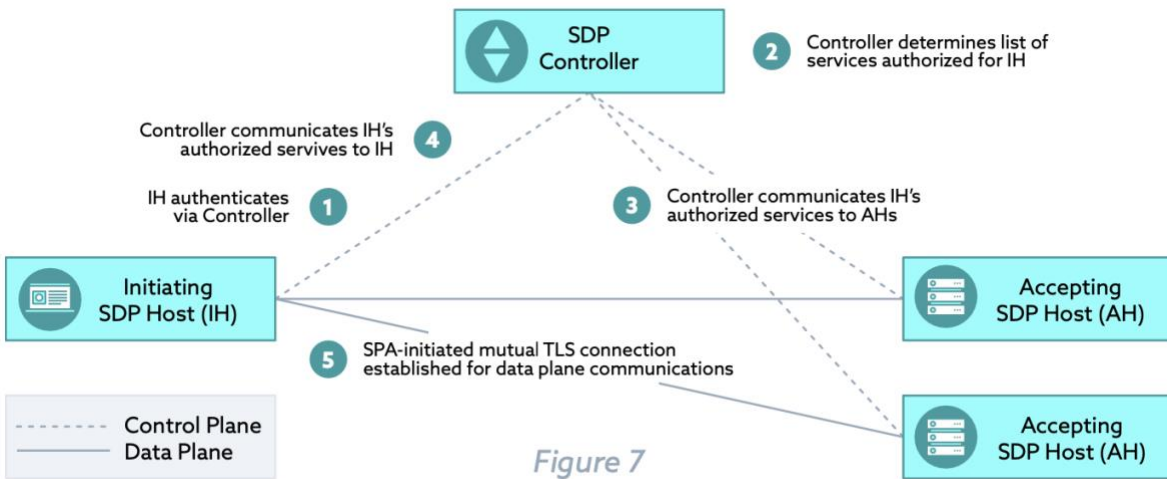
Figure 6 - Initiating Host Onboarding Workflow

## 액세스 워크 플로우

IH 는 표 1 과 그림 7 의 단계에 따라 SDP 시스템에 의해 보호되는 워크로드(리소스)에 접속하기 위한 액세스 워크 플로우를 시작한다. 액세스 플로우는 SDP 의 모든 컴포넌트(IH, 컨트롤러 및 AH)를 포함한다.

| 단계 | 액세스 워크 플로우                                                                                                                |
|----|---------------------------------------------------------------------------------------------------------------------------|
| 1  | On-boarding 된 IH 가 온라인으로 돌아오면(예로, 디바이스의 재부팅 후 또는 사용자가 연결을 시작할 경우 등) SDP 컨트롤러에 접속하여 인증한다.                                  |
| 2  | SDP 컨트롤러는 해당 ID 제공자를 통해 IH 를 인증한 후 (AH 를 통해) IH 와 통신할 수 있는 서비스 목록을 결정한다. 컨트롤러는 이 경우 해당 목록을 IH 에 전달하지 않으며, 3 단계 이후까지 대기한다. |
| 3  | SDP 컨트롤러는 AH 가 IH 로부터의 통신과 양방향 암호화된 통신에 필요한 사용자, 디바이스 및 서비스 간 연결을 정의하는 모든 정보를 받아들이도록 지시한다.                                |
| 4  | SDP 컨트롤러는 IH 에 인증된 AH 및 서비스 목록과 양방향 암호화된 통신에 필요한 정보를 제공한다.                                                                |
| 5  | IH 는 SPA 프로토콜을 사용하여 SPA 의 정보를 확인하기 위해 승인된 각 AH 와의 접속을 시작한다. 이후 IH 는 이러한 AH 에 대한 양방향 TLS 연결을 생성한다.                         |

Table 1: Access Workflow

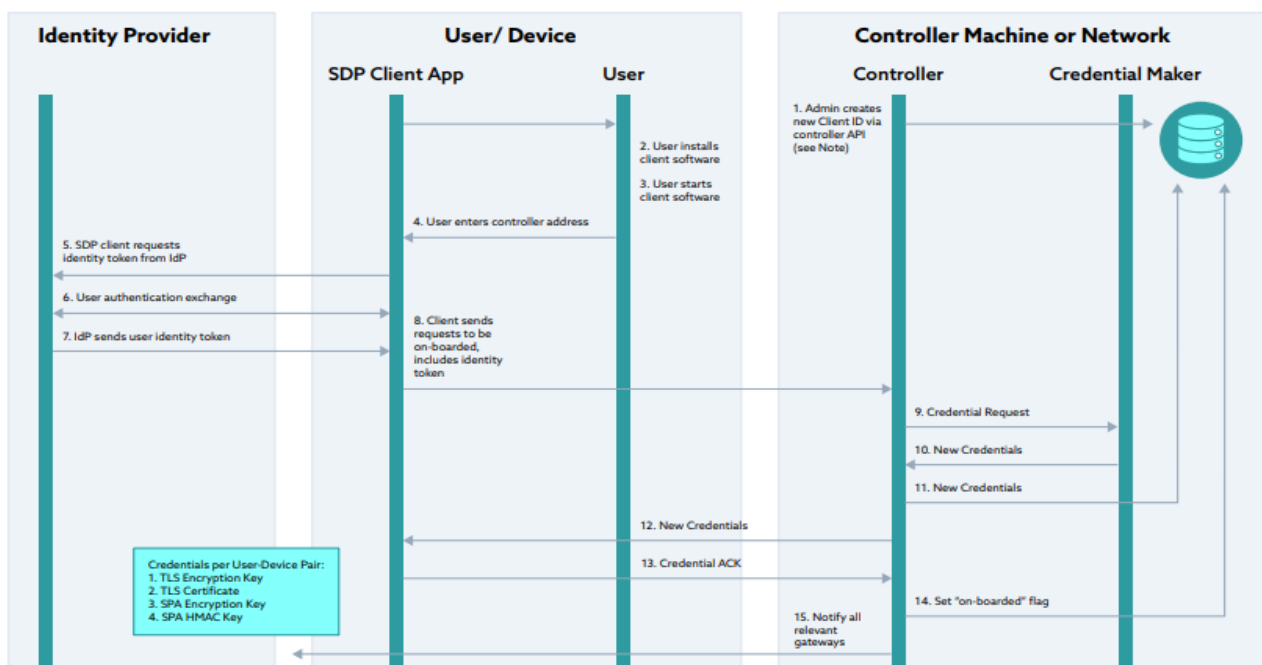


## IH On-boarding 워크 플로우 예

SDP 컨트롤러, IH, AH, 사용자의 on-boarding은 그림 2에 나타난 배치 모델이나 구현 방법에 따라 다르다. SDP 시스템의 설정 및 운영은 API(Application Programming Interface) 나 관리용 UI를 통해 수행될 수 있다.

위 워크 플로우에서 설명한 바와 같이 SDP 시스템이 배치되고, 구성되어 SDP 컨트롤러 및 AH가 온라인으로 전환된다.

다음 순서는 외부 IdP(Identity Provider)에서 인증하는 IH의 사용자의 경우 on-boarding 워크 플로우 예<sup>7</sup>가 된다.



<sup>7</sup> Based upon SDP Open Source Reference Implementation - <https://www.waverleylabs.com/open-source-sdp/>  
© Copyright 2022, Cloud Security Alliance. All rights reserved.

참고 – 이 예에서는 사용자 등록을 위해 IdP 정보를 사용하고 있다. 등록 시스템은 등록 프로세스의 일부로, MFA 나 디바이스 속성 검증(예로, 기업에 배치된 인증서 또는 End point 관리 소프트웨어) 등 다중 인증 요소가 포함될 수 있다. 또한 이 예에서는 주요 키 정보(SPA 키나 다른 공유 secret 등)의 배포도 생략한다. 예로, 오픈소스의 SDP 구현은 이러한 핵심 정보를 생성한 다음 기업이 안전한 방식으로 SDP 컴포넌트에 전송하는 것도 가능하게 된다.

이 샘플 워크 플로우에서, IH 는 클라이언트(그림의 SDP 클라이언트)를 사용하여 IdP(Identity Provider)와 협상(negotiate)한다. 일부 SDP 모델의 경우 SDP 컨트롤러가 IdP 와 신뢰 관계를 유지하여 클라이언트가 SAML 토큰으로 전달했는지 검증한다.

## Single Packet Authorization (SPA)

SDP 핵심 원칙 중 하나는 워크로드(리소스)와 SDP 인프라 자체에 권한이 없는 엔티티가 액세스할 수 없어야 한다는 것이다. 이는 엔티티가 모든 SDP 컴포넌트에 접속할 때 암호화하여 인증되는 것을 의무화하도록 구현한다. 이 경우 인증되지 않은 엔티티는 SDP 컴포넌트와 네트워크 연결을 설정할 수 없으므로 취약성을 악용하거나, 로그인에 무차별 공격하거나, 도난 당한 사용자 인증 정보를 사용할 수 없게 된다. 이는 인터넷 상의 모든 악의적인 행위에 처해 있는 일이 많은 VPN 과 같은 기존 원격 액세스 솔루션과 대조적이다.<sup>8</sup>

SDP 가 이를 위해 사용하는 메커니즘은 SPA(Single Packet Authorization)이다. SPA 의 SDP 버전은 아래에서 설명하는 것과 같이 SPA 패킷에 포함된 RFC 4226 HMAC 기반 원 타임 패스워드 「HOTP」<sup>9</sup>를 기반으로 한다.

SDP 는 규격상 컨트롤러나 AH 에 대한 액세스를 보호하기 위해 SPA 를 사용하는 것을 중심으로 하고 있으나, 적절한 아키텍처에서 동작할 수 있는 신뢰할 수 있는 대체 수단도 있다. 다음의 SPA 대체 수단의 절을 참고하기 바란다.

SPA 가 SDP 내에서 달성하는 주요 원칙은 다음과 같다.

- **SDP 시스템 컴포넌트 은폐:** 컨트롤러와 AH 는 원격 시스템으로부터의 접속 시도에 대해 다음과 같이 SDP 시스템에 대해 유효한 SPA 패킷이 제공되지 전까지 응답하지 않는다. 특히 UDP 기반 SPA 의 경우 호스트가 TCP SYN 에 응답하지 않으므로 잠재적인 공격자에게 정보가 노출되는 것을 방지할 수 있다(디폴트 드롭 방화벽은 이를 구현하는 방법의 한 예이다). 이는 Standalone AH(SDP 게이트웨이)와 서버/워크로드의 논리적 일부인 AH 의 양쪽에 해당한다.

<sup>8</sup> to be fair, there are some commercial and open source VPN-like solutions that do hide the infrastructure. Wireguard is one open such example.

<sup>9</sup> See <https://tools.ietf.org/html/rfc4226>

- **TLS의 서비스 거부 공격 완화:** 인터넷에 연결된 HTTPS(TLS를 사용) 프로토콜을 실행하고 있는 서버는 서비스 거부(DoS)나 분산형 서비스 거부(DDoS) 공격에 매우 취약하다. SPA를 사용하면 서버가 TCP 또는 TLS 연결을 설정하는 오버헤드를 발생시키기 전에 무단 연결 시도를 신속하게 거부할 수 있어 인증된 연결을 허용할 수 있으므로 이러한 공격을 완화할 수 있다.
- **공격 탐지:** 다른 호스트에서 컨트롤러 또는 AH로 전송되는 최초의 패킷은 SPA 패킷이어야 한다. AH가 다른 패킷을 수신할 경우 공격으로 간주해야 한다. 따라서 SPA를 통해 SDP는 단일 악성 패킷을 기반으로 공격을 판단할 수 있다.

설계상 수신하는 SPA 패킷의 검증은 계산이 가벼워 DDoS 공격에 대한 SDP 시스템의 복원력을 향상시킨다(SDP Cloud Security Alliance, SDP as a DDoS Defense Mechanism<sup>10</sup> 백서 참조).

SPA는 IH-컨트롤러, AH-컨트롤러 그리고 IH-AH의 통신을 시작한다. SPA 패킷은 구현 방식에 따라 UDP 또는 TCP 프로토콜을 사용하여 시작한다.

UDP 기반 SPA는 SPA로 보호된 서버에 위에 나열된 모든 보안 이점을 제공한다. TCP 기반 SPA는 UDP 기반 SPA보다 낮은 수준으로 이러한 이점 중 일부를 제공한다. 구체적으로 TCP 기반 SPA를 사용하는 SDP 컴포넌트는 모든 원격 사용자(악의적 행위를 할 가능성이 있는)에게 오픈 포트를 공개하므로, 서버(SDP 게이트웨이, AH)는 은폐되지 않는다. 또한, 서버는 부분적으로 DDoS 공격의 대상이 된다. 원격 IP 주소에서 TCP 접속을 허가하여, TLS 연결을 만들기 전에 SPA 유효성 검사를 수행한다. TCP 접속은 TLS 접속보다 리소스 사용량이 적지만 서버 리소스를 사용하여 서버를 DDoS의 위험에 처하게 한다. 결과적으로 TCP 기반 SPA를 사용할 경우 TCP는 서버가 잘못된 SPA 패킷을 기반으로 공격을 탐지할 수 있으나, TCP 접속이 설정된 후에 한하여 서버 리소스를 사용하게 된다.

서버 노출을 최소화하기 위한 또 다른 고려사항은 AH 및 컨트롤러의 DNS 배열이다. IP를 통해 직접 접속하거나, Private DNS 서비스만을 경유할 경우 공격자로부터의 서비스의 가시성을 저하시킬 가능성이 있다. 즉, SDP 컴포넌트의 public DNS 엔트리(controller1.sdp.mycompany.com 등)의 존재 자체가, 악의적 행위자의 공격 대상이 될 가능성이 있다. 예로, SDP 인프라에 대한 대규모 DDoS 공격 등이 있다.

아래 권장 SPA 메시지 포맷은 프로토콜의 보안 및 복원력을 향상시키기 위해 SDP 규격의 v1 이후 업데이트되었다.

## SPA 메시지 포맷

SPA 메시지 포맷은 SDP 구현마다 다를 수 있으나, 모든 SDP 시스템은 컴포넌트 간의 접속을 시작하기 위한 메커니즘으로 SPA를 지원해야 한다. SPA를 사용하기 위해 각 SPA 패킷이 유효한 SPA 패킷을 구성하기 위해 공유 secret를 요구하므로 SPA 패킷 생성자와 수신자가 공유 신뢰 기점을 갖춰야 한다.

<sup>10</sup> <https://cloudsecurityalliance.org/artifacts/software-defined-perimeter-as-a-ddos-prevention-mechanism/>  
© Copyright 2022, Cloud Security Alliance. All rights reserved.

이러한 신뢰 기점, 즉 공유 secret 이 SDP 컴포넌트에 안전하게 전달되는 방법은 구현에 따라 다르며 이 규격의 범위를 벗어난다. 일반적으로 이러한 정보는 IH 및 AH 의 on-board 프로세스에 포함된다.

|                   |                                                                                                                                                                                                                                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ClientID          | 256 비트 숫자 식별자로, 사용자-디바이스 쌍(pair)별로 할당된다. 이 필드는 패킷을 보내는 사용자, 디바이스 또는 논리적 그룹을 구별하는 데 사용된다.                                                                                                                                                                                                        |
| Nonce             | 16 비트 랜덤 데이터 필드는 SPA 패킷 재사용을 방지하여 재생 공격(replay attack)을 방지한다.                                                                                                                                                                                                                                   |
| Timestamp         | 짧은 유효 기간(예: 15~30 초)을 보장하여 오래된 SPA 패킷의 서비스를 방지한다. 또한 수신자에게 필요한 재생(replay) 탐지 캐싱을 줄일 수 있는 메커니즘을 제공한다.                                                                                                                                                                                            |
| Source IP Address | Initiating host 의 public 으로 공개되어 있는 IP 주소이다. 이는 수신 호스트가 패킷 헤더 중 소스 IP 주소에 의존하지 않도록 포함되며, 이 주소는 도중에 쉽게 수정된다. IH 는 AH 가 패킷의 발신지로서 사용할 IP 주소를 가져올 수 있어야 한다.                                                                                                                                        |
| Message Type      | 이 필드는 선택 사항으로, 연결이 설정된 후 IH 에서 예상할 메시지 유형을 수신자에게 알리는 데 사용할 수 있다.                                                                                                                                                                                                                                |
| Message String    | 이 필드는 선택 사항으로, 메시지 유형 필드에 따라 상이하다. 예를 들어, 이 필드를 사용하여 연결 시 IH 가 요청할 서비스를 지정할 수 있다.                                                                                                                                                                                                               |
| HOTP              | 해시화 된 원타임 암호(해시 OTP)는 공유 secret 을 기반으로 RFC4226 에서 설명한 알고리즘에 의해 생성된다. 신뢰성을 설정할 경우 SPA 패킷에서 OTP 를 사용해야 한다. SPA 패킷의 신뢰성을 제공한다고 하는 포괄적인 목표가 있을 경우 다른 OTP 알고리즘으로 대체할 수 있다.                                                                                                                           |
| HMAC              | 위 모든 필드를 대상으로 계산된다. 알고리즘은 SHA256(권장), SHA384, SHA512, SM3, Equihash 또는 기타 효율적이고 강력한 알고리즘 등을 선택할 수 있다. HMAC 은 공유(secret) 시드(seed)를 사용하여 계산된다. HMAC 은 메시지의 모든 선행 필드에 대해 계산되어, AH 에서 메시지 무결성을 검증하는 데 사용된다. HMAC 검증은 계산적으로 경량이기 때문에 DoS 공격에 대한 복원력을 제공하는 데 사용할 수 있다. 잘못된 HMAC 이 있는 SPA 패킷은 즉시 파기된다. |

Table 2: SPA Message Scheme

다른 SPA 옵션에는 IH 의 개인 키(부인 방지용) 또는 AH 의 공개 키(기밀 유지용)를 사용한 추가 암호화가 포함될 수 있다. 그러나 비대칭 암호화는 계산량이 많으며, 수신자가 DoS 공격에 대한 AH 의 복원력을 유지하기 위해 단순한 HMAC 와 같이 경량 검증 메커니즘 후에 한하여 사용해야 한다.

## 안전한 자기 완결형 무연결형 메시지 전송 프로토콜, SPA

SDP 시스템 내 SPA의 흥미로운 "사이드" 사용 사례 중 하나는 SPA 패킷을 연결을 시작하기 위한 수단뿐만 아니라 원격 element에서 데이터를 송신하기 위한 수단으로 사용할 수 있다. SPA 패킷은 공유 secret을 기반으로 하기 때문에 수신자는 SPA 패킷에 포함된 데이터가 유효한 SDP 클라이언트에 의해 발급되었음을 신뢰할 수 있다.

SPA 시드가 지정된 클라이언트 고유한 경우(SPA 패킷 내 ClientID에서 식별), Message String 필드를 사용하여, 클라이언트가 의미 있는 데이터를 송신할 수 있다. 이 경우, 추가 처리 또는 정책 평가가 필요하지 않게 되며 TCP 또는 TLS 연결을 설정할 필요가 없다.

이는 예를 들어 소량의 데이터를 정기적으로 전송해야 하는 분산형 IoT 센서에 유용하다. SPA 패킷에 데이터를 포함할 경우 이러한 디바이스가 TCP 및 TLS 연결을 설정하는 데 드는 오버헤드 없이 이 작업을 수행할 수 있다. 물론 이러한 메커니즘에는 몇 가지 단점이 있다. 수신자(AH)는 이 데이터를 기대하고 있어야 하며, 이 데이터는 단방향 전송이기 때문에 'fire and forget' SPA 패킷 전송과 함께 데이터가 실제로 수신되었다는 확인을 수신하지 않는다. 그럼에도 이러한 방법은 데이터가 비중요 정보인 경우 일부 환경에 SPA를 적용하는 데 유용한 방법이다.

## SPA 대안

SDP 아키텍처는 유연성을 염두에 두고 설계되었으며, 이번 규격 개정에서는 생성될 수 있는 접근 방식과 배치 모델의 다양성을 수용한다. 예를 들어 SDP는 각각 다른 문제를 해결하는 데 도움이 될 수 있는 6가지 배치 모델을 정의하고 있다, 각각의 모델은 다른 문제 해결에 적합하며, 다른 방식으로 구현되어 다른 tradeoff를 만들어 낼 가능성이 있다. SPA는 인프라를 은폐, DDoS에 대한 복원력 제공이라는 SDP의 핵심 원칙을 달성하기 위한 안전한 메커니즘을 제공한다. 공격 및 공격 탐지 기능을 향상시킨다. SPA는 SDP를 자체 포함 시스템으로 만들 수 있다는 이점도 있다. 시드된 IH는 외부 시스템에 의존하지 않고 컨트롤러 및 AH와 안전하고 안정적으로 암호화된 통신을 설정할 수 있다.

하지만, SPA는 시스템이 이러한 목표를 달성할 수 있는 유일한 메커니즘은 아니다. 여기에서 정의한 원칙을 제공하는 대체 접근도 있으며, SDP 및 제로 트러스트 원칙을 지원하는 시스템의 일부가 될 수 있다. 예를 들어, SPA가 없는 SDP 시스템은 SDP의 컨트롤러에 대한 제어 채널을 통해 전 세계적으로 액세스할 수 있는 엔터프라이즈 IdP를 활용하여 SDP 컨트롤러와 AH에 제어 채널을 제공할 수 있다. 이 모델에서 IdP에 대한 IH의 인증은 SDP 컨트롤러 및 AH에 대한 제어 플레인 메시지를 트리거하여 IH의 인증이 성공하여, IH의 Public IP 주소로부터의 즉시 연결을 기대한다. IH는 컨트롤러 및 AH에 대한 TCP 연결을 설정할 수 있으며, TCP 연결이 된 후(상호 인증된 TLS(mTLS)) 및 아래에 정의된 다른 보안 계층이 뒤따르게 된다.



물론, 모든 시스템과 아키텍처는 자체적인 tradeoff 가 있다. 이 접근 방식이 위에서 언급한 3 가지 원칙을 달성하는 한 SPA 의 대안을 사용하는 시스템은 안전하고 가치 있는 SDP 시스템의 일부가 될 수 있다.

## 컴포넌트 간 전송 계층 상호 인증

SDP 는 연결 설정 프로세스 전반에 걸쳐 여러 계층 검증이 필요하다. 최초의 단계는 앞에서 설명한 바와 같이 SPA 이다. 이 절의 주제인 다음 단계는 분산 SDP 컴포넌트 간의 암호화된 보안 연결 설정의 일부로서 상호 인증에 대한 요구사항이다. 디바이스 및 사용자 검증을 포함한 추가 단계는 다음에서 설명한다.

컴포넌트가 SPA 를 사용하여 인증 및 인가된 후 SDP 시스템의 주요 컴포넌트 간의 접속에는 TLS 또는 Internet Key Exchange(IKE) 등의 다른 대체 수단을 사용할 필요가 있다. 디바이스를 SDP 의 인증된 구성원으로 검증해야 한다. 특히 IH-AH, IH-컨트롤러, 컨트롤러-AH 의 연결은 상호 인증을 사용해야 한다. TLS 는 TCP 및 UDP 에서 지원되며, SDP 또한 TCP 및 UDP 모두 허용하고 있다.

상호인증을 지원하지 않는 취약한 암호 스위트(cipher suite) 및 프로토콜은 부적절하며, 안전하지 않다. 강력한 암호와 프로토콜은 각 컴포넌트가 신뢰할 수 있는 기관으로부터 발행된 유효한 비밀 키를 포함하고 있는지를 확인할 수 있으며, 양방향으로 서명된 인증서를 확인하여 중간자 공격(MITM)<sup>11</sup>의 가능성을 대폭 감소시킬 수 있다.

이러한 컴포넌트의 루트 인증서는 엔터프라이즈 PKI 시스템 또는 SDP 관련 CA 가 되어야 한다. 소비자용 브라우저에 관련된 사전 발급 또는 암묵적으로 신뢰할 수 있는 인증서에 의존해서는 안 된다. 이 인증서는 공격자가 침해당한 인증 기관에서 인증서를 위조하는 위장 공격의 대상이 된다. SDP 는 OCSP(Online Certificate Status Protocol)<sup>12</sup> 또는 다른 메커니즘을 사용하는 등 효력을 잃은 인증서를 효율적으로 탐지할 수 있도록 하는 체계를 구현해야 한다.

SDP 시스템이 사용하는 전송 계층 접근 방식은 MITM TLS 다운그레이드 프로토콜 공격(mTLS 는 회피) 등 잠재적인 공격에 대해서는 복원력이 있어야 한다.

## 디바이스 검증

상호 전송 계층 인증은 SDP 에 대한 액세스를 요청하는 디바이스가 만료되지 않거나 취소되지 않는 개인 키를 보유하고 있음을 증명하지만 디바이스가 보안 또는 구성 요구사항을 충족하는지 검증하지 않는다.

<sup>11</sup> <https://wott.io/blog/tutorials/2019/09/09/what-is-mtls>

<sup>12</sup> <https://tools.ietf.org/html/rfc6960>

디바이스 검증의 목적은 디바이스가 보안 요구사항을 충족하는지 증명하는 것이다. 엔터프라이즈 환경에서 컨트롤러는 통제되는 환경에 존재하므로, 신뢰할 수 있는 디바이스로 간주된다.<sup>13</sup> SDP 게이트웨이는 SDP 를 운용하는 기업의 통제 하에 있는 경우 신뢰할 수 있는 구성요소로 간주된다. 즉, 이러한 구성요소는 통제되고 관리되는 환경에서 실행되며, 이러한 컴포넌트는 엔터프라이즈 변경 관리 제어 및 구축 프로세스를 따라야 하며 의도적으로 SDP 시스템에 on-boarding 되어야 한다. 컨트롤러 및 게이트웨이 컴포넌트의 경우 디바이스의 검증은 선택 사항으로 간주되어야 한다.

SDP 를 통해 인증 및 인증되지 않는 경우 모든 디바이스에서 패킷이 손실되므로 인증되지 않은 디바이스에서 들어오는 패킷은 차단된다. 디바이스 유효성 검사 프로토콜과 세부사항은 기업 및 제품별로 다르므로 SDP 사양의 범위를 벗어나지만 향후 과제의 관심 분야이다.

SDP 시스템은 엔터프라이즈 디바이스 관리/End point 관리 시스템과 통합할 수 있는 기능을 지원하고 디바이스 보안 현황 점검을 디바이스 유효성 검사 프로세스에 포함해야 한다. 또한 SDP 시스템은 SDP 가 사용자 디바이스에서 로컬로 실행되는 소프트웨어를 가진 환경에서 로컬 디바이스 보안 현황 점검을 수행할 수 있는 기능을 지원해야 한다. 예를 들어, SDP 클라이언트는 mTLS 인증에 사용될 수 있는 기업에서 발급한 유효한 인증서가 사용자 디바이스에 포함되어 있는지 확인할 수 있다. 또는 디바이스에서 승인된 백신 컴포넌트를 실행 중인지 확인할 수 있다. 이러한 측면은 SDP 가 지원하는 전반적인 제로 트러스트 접근 방식과 관련이 있다. 즉, 디바이스 정보를 액세스 정책 결정을 위한 추가 컨텍스트로 사용하는 것입니다. 또한 “디바이스”는 검증해야 하는 서버를 가리킬 수도 있으며, 사용자 디바이스와 동일한 여러 가지 방법으로 검증하는 것이 가능하며 검증할 필요가 있다.

## SDP 와 IoT 디바이스

NIST 800-207 제로 트러스트의 핵심 원칙 중 2 가지는 “모든 데이터 소스와 컴퓨팅 서비스” 및 “모든 통신”<sup>14</sup>에 대한 액세스를 보호해야 한다고 명시하고 있으며, 이는 IoT 디바이스도 해당한다. IoT 는 대부분의 IT 환경(예로 프린터, IP 카메라, VOIP 전화)에 존재하는 디바이스를 포함하여 환경 또는 의료 센서 그리고 산업 제어 시스템과 같은 보다 전문화된 디바이스를 포함한다.

이러한 디바이스는 외부에 공개되어 있어, 일반적인 SDP IH 로 작동하기 위해 완전한 기능의 소프트웨어 설치 및 작동을 지원할 수 있는 IH 로 동작하는 일반 호스트와 동일하게 처리될 수 있다. 여기서 설명하는 내용은 3 가지 카테고리 중 하나에 해당하는 디바이스로 한정된다. 이 디바이스는 폐쇄형(확장 불가) 디바이스로, 기업은 추가 소프트웨어를 설치할 수 없고, 범용 컴퓨터(예로 아두이노 등)이나 컴퓨터/메모리/스토리지의 용량이 제한되거나, SDP 클라이언트 소프트웨어를 구축하지 않는

<sup>13</sup> 비밀키 및 SPA secret 은 안전하게 보관하여, 정기적으로 교체할 필요가 있다. 이것은 앞으로의 과제이다.

<sup>14</sup> See NIST Special Publication 800-207, Zero Trust Architecture, Page 6

것을 선택한 모든 기능을 작동하는 범용 시스템이다. 모든 경우, 이러한 디바이스는 네트워크 장비이며, 이러한 장비와의 액세스는 SDP 아키텍처 및 정책 모델에 포함되어야 한다.

확장이 불가능한 디바이스가 SDP 시스템에 포함되기 위해서는 IoT 디바이스 네트워크 트래픽을 캡처하고 이를 SDP 시스템으로 브로커하기 위한 “업스트림” 네트워크 장비(“브로커” 또는 “커넥터”)가 필요하다. SDP 브로커는 IoT 디바이스를 대체하여 논리적 IH 역할을 한다. 이 브로커는 IoT 디바이스로 송수신되는 트래픽을 모두 제어할 수 있어야 한다.

용량에 제한이 있는 디바이스인 경우 SDP 소프트웨어 일부를 실행할 수 있으나 전체 기능을 갖춘 IH로 동작할 수 없다. SDP는 이러한 디바이스 및 잠재적으로 이러한 디바이스에 대한 통신을 위한 대체 모델에 한하여 개방하고 있다. 예를 들어, 저전력, 저용량의 환경 센서는 데이터를 안전하게 전송하기 위해 유효한 SPA 패킷만 생성하는 경량 SDP 클라이언트를 실행할 수 있다. TLS 연결을 설정하거나 인증이 불필요하기 때문에, 계산량이 적은 방법으로 데이터를 안전<sup>15</sup>하게 전송할 수 있다.

이러한 아이디어들은 제로 트러스트와 IoT에 관한 향후 연구의 좋은 출발점이 되며, 이 분야에 있어 새로운 연구와 출판을 기대하고 있다.

## 액세스 정책

제로 트러스트는 보호된 리소스에 대한 액세스를 제어하는 액세스 정책 아이디어에 중점을 두고 있다. 제로 트러스트의 주요 개념인 정책 결정 포인트와 정책 시행 포인트를 상기한다. NIST 제로 트러스트 아키텍처 문서에 따르면, 핵심 원칙 중 하나로 “리소스에 대한 액세스는 클라이언트의 신원, 애플리케이션/서비스 및 요청 자산 상태를 포함한 동적 정책에 의해 결정되어, 다른 행위 및 환경 속성을 포함할 수 있다.”<sup>16</sup>고 명시되어 있다. 또한 CISA (Cyber Security & Infrastructure Security Agency) 제로 트러스트 성숙도 모델에서는 정책이 기존 성숙도 레벨에서 정적인 것에서 고급 수준에서 교차 도메인 입력 및 출력을 사용하는 것으로 최적 수준에서 “자동/관찰된 트리거에 기반한 동적 정책”<sup>17</sup>으로 전환되는 방법에 대해 설명한다.

<sup>15</sup> 전송 시 데이터가 암호화되지 않으므로 메시지 무결성은 제공되나, 개인 정보 보호는 제공하지 않는다(별도 배포된 키로 암호화될 수 있음).

<sup>16</sup> See NIST Special Publication 800-207, Zero Trust Architecture, Page 6

<sup>17</sup> US Cybersecurity and Infrastructure Security Agency Cybersecurity Division, Zero Trust Maturity Model, Pre-decisional Draft, June 2021, Version 1.0, Page 5.

그러나, 제로 트러스트 아키텍처나 기업 요건이 다양하므로 효과적이나 광범위하게 적용할 수 있는 정책 모델 프레임워크와 관련 어휘를 정의하는 것은 어렵다. 예를 들어, SDP의 경우 65 가지 서로 다른 배포 모델이 있으며, 각 모델 적용 기능은 다르다. 이 범위의 결과로 초기 제로 트러스트 문서는 정책(예, SDP 규격서 버전 1)에 대해서 언급하지 않았다. 새로운 제로 트러스트 모델은 정책에 어떻게 접근하는가에 대해 몇 가지 지침을 제공한다.

NIST 800-207은 그림 9에 개념적으로 묘사한 신뢰 알고리즘을 평가하는 정책 엔진의 개념을 도입하고 있다.

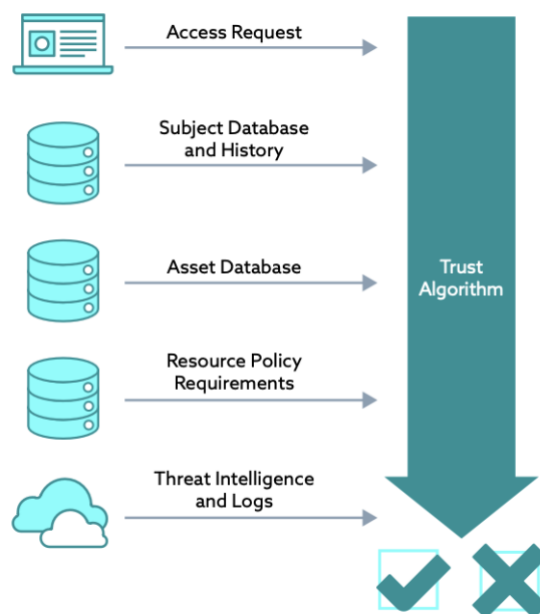


Figure 9: Trust Algorithm figure from NIST 800-207

NIST 800-207에서는 리소스 정책 요구사항에 대해 설명하는 정책 모델을 설명한다.

“이러한 정책은 사용자 ID와 속성 데이터베이스[SP800-63]를 보완하여 리소스에 대한 액세스를 위한 최소 요구사항을 정의한다. 요구사항에는 MFA 네트워크 위치(예, 해외 IP 주소로부터의 액세스 거부), 데이터 기밀성 및 자산 구성 요청과 같은 인증 보증 레벨이 포함될 수 있다. 이들의 요건은 custodian(즉, 데이터 책임자)와 데이터를 이용하는 비즈니스 프로세스의 책임자(즉, 미션 책임자)의 양방에 따라 책정되어야 한다.”

CISA 제로 트러스트 성숙도 모델은 정책에 대해 직접 언급하지 않지만, 정책에서 표현 가능한 여러 성숙도 레벨의 시스템 기능 예를 제시하여 간접적으로 정책을 표현하고 있다.

보편적으로 적용되는 제로 트러스트 정책 모델에 대한 구조와 어휘를 정의하는 것은 어렵다. 다만 이는 보안 철학으로 제로 트러스트 본질에 내재되어 있다. SDP 아키텍처는 보다 구체적이며, SDP 구축 모델마다 차이가 있으나 구조화된 정책 모델에 적합하다고 생각한다. 예를 들어, 디바이스에 기업이 발급한 인증서가 포함되어 있는지 확인해야 하는 디바이스 현황 검사 정책을 생각해볼 때, 이는 사용자의 디바이스 상에서 로컬 에이전트가 실행되고 있는 시스템에는 쉽게 실행할 수 있으나 에이전트가 없는 시나리오에서는 더 어렵다.<sup>18</sup> 이러한 유형의 실 제약은 보다 일반적인 정책 측면(예, NAT)로 제한되는 업계 조사와 지침 문서(필요에 따라 구현 중립적이어야 함)를 초래한다. 구체적이고 상세한 정책 모델이 아닌 기업 보안 현황 점검에 적합하다.

제로 트러스트 정책 모델은 이러한 v2 SDP 규격 범위 밖이나, SDP 및 ZT Working Group을 포함하여 업계 전반에서 향후 연구를 수행할 수 있는 분야이다. 이 주제에 대한 향후 협업 및 출판을 기대한다.

<sup>18</sup> 또한 일부 운영 체제에서는 에이전트 없이 실행할 수 없는 경우도 있다.

# SDP 프로토콜

SDP 프로토콜은 AH-컨트롤러 프로토콜, IH-컨트롤러, IH-AH 프로토콜, 로깅의 4 가지 절로 구성되어 상세하게 설명한다. 여기에 표시된 SDP 프로토콜은 SDP 구성요소 간의 SPA 를 사용한 통신을 나타내고 있으며, 고정 또는 표준화된 메시지 형식을 규정하는 것은 아니다. SDP 배치 모델마다 다른 상호 작용과 메시지가 필요하다. 이 문서의 목표는 고정된 또는 표준화된 메시지 형식을 규정하는 것이 아닌 동작하는 대표적인 시스템을 보여주는 것이다.

앞에서 설명한 것과 같이, 컴포넌트 또는 디바이스 on-boarding 을 위한 메시지 그리고 데이터 플로우에 포함되지 않는 것에 주의해야 한다.

여기서 설명하는 프로토콜은 시간 초과, 재시도 또는 기타 오류 처리 메커니즘을 지정하지 않는다. 또한 IH 로그인 프로세스 중 컨트롤러가 IH 를 인증하기 위해 엔터프라이즈 ID 관리 시스템에 추가 외부 호출을 할 수 있다. 마지막으로 아래 예제는 컴포넌트 사이에서 TCP 및 TLS 접속을 사용하고 있는 것에 주의해야 한다. 앞에서 설명한 바와 같이, connectionless 의 UDP 기반의 프로토콜(즉, DTLS)를 사용하는 것도 가능하나, 명확성을 위해 아래 예에서는 설명하고 있지 않다.

# AH-컨트롤러 프로토콜

## AH 와 컨트롤러 간 시퀀스 다이어그램

AH 가 컨트롤러에 연결하기 위한 프로토콜 시퀀스 다이어그램은 그림 10 에 나타낸다. UDP 기반 SPA 패킷은 SDP 컨트롤러를 무단 액세스로부터 보호하기 위한 최초의 패킷이다.

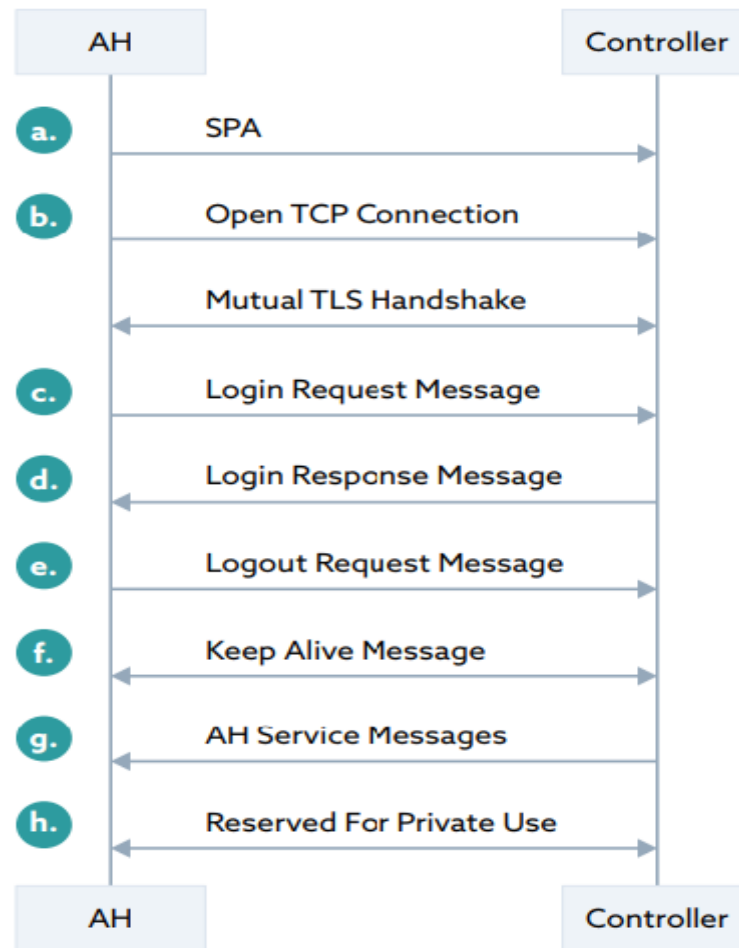


Figure 10: Accepting Host connects to the Controller

다음에서는, AH 와 컨트롤러 사이에서 주고받는 다양한 메시지 또는 포맷을 정의한다. 기본적인 메시지 포맷은 다음과 같다.

|                 |                                                 |
|-----------------|-------------------------------------------------|
| Command (8-bit) | Command specific data (command specific length) |
|-----------------|-------------------------------------------------|

### a. SPA

SPA 패킷은 AH 가 컨트롤러에 접속을 요구하기 위해, 앞에서 설명한 포맷에 따라 전송된다.

## b. TCP 접속 및 상호 인증된 통신 설정

SPA 패킷을 보낸 후 IH는 컨트롤러 상의 TCP 접속의 확립을 시도한다. 컨트롤러가 SPA 패킷이 유효하다고 판단한 경우 해당 TCP 연결을 허용한다. 이후 mTLS 연결 설정에 필요한 상호 인증이 행한다. UDP 기반 DTLS의 경우, UDP는 connectionless 형 프로토콜이므로 논리적인 접속이 된다.

## c. 로그인 (SDP join) 요청 메시지

로그인 요청 메시지는 AH가 동작 중인 것을 의미하며, active한 AH로서 SDP에 join하는 것을 요청하기 위해 AH로부터 컨트롤러에 송신된다. AH의 로그인 요청에는 AH가 컨트롤러에 대해 자신을 식별 및 인증하기 위한 증명서를 포함할 수 있다.

|      |                          |
|------|--------------------------|
| 0x00 | No command-specific data |
|------|--------------------------|

## d. 로그인 (SDP Join) 응답 메시지

로그인 응답 메시지는 컨트롤러에 의해 전송되어 로그인 요청이 성공했는지 여부를 나타내고, 성공한 경우 AH 세션 ID를 제공한다. 컨트롤러가 AH의 로그인 요청을 거부할 수 있다. 예를 들어, 자격 증명에 잘못되었기 때문일 수 있다. 또는 컨트롤러가 시스템 라이선스 또는 동시 접속 수 제한 등을 실시하여 AH의 로그인을 거부할 가능성도 있다.

두 경우 모두 연결이 거부되고 컨트롤러가 AH에서 TCP 연결을 종료한다.

|      |                       |                                                                                    |
|------|-----------------------|------------------------------------------------------------------------------------|
| 0x01 | Status Code (16 bits) | AH Session ID (256 bits) and optionally, renewed SPA keys and TLS keys for the AH. |
|------|-----------------------|------------------------------------------------------------------------------------|

상태 코드는 구현에 따라 다르다. AH 세션 ID는 시스템 로그 내에서 사용된다. 이를 나타내는 JSON 규격은 다음과 같다.

| Format                                                                                                                                                                                                                                                                         | Example                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>{  "session_id" : &lt;256 bits&gt;,<br/>  "credentials":<br/>    ["spa_encryption_key":&lt;64 bit&gt;,<br/><br/>    "spa_hmac_key": &lt;64 bit&gt;,<br/><br/>    "tls_key": &lt;file contents&gt;,<br/><br/>    "tls_cert" &lt;file contents&gt;<br/><br/>  ]<br/>}</pre> | <pre>{  "session_id": "0x1234..."<br/>  "credentials":<br/>    ["spa_encryption_key":"aldskf...",<br/><br/>    "spa_hmac_key": "asldjfk...",<br/><br/>    "tls_key": "tls_key",<br/><br/>    "tls_cert": "tls_cert"<br/><br/>  ]<br/>}</pre> |

## e. 로그아웃 요청 메시지

로그아웃 요청 메시지는 AH 가 컨트롤러로 전송하는 것으로, AH 를 더 이상 사용할 수 없으며 컨트롤러의 다른 메시지를 수신할 수 없음을 나타낸다. 컨트롤러에서 보낸 응답 메시지는 없으며 TLS 및 TCP 연결은 AH 또는 컨트롤러에 의해 종료되어야 한다.

|      |                          |
|------|--------------------------|
| 0x02 | No command-specific data |
|------|--------------------------|

## f. Keep-Alive 메시지

Keep-Alive 메시지는 AH 또는 컨트롤러에 의해 전송되어 현재 active 상태임을 나타낸다.

|      |                          |
|------|--------------------------|
| 0x03 | No command-specific data |
|------|--------------------------|

## g. AH 서비스 메시지

서비스 메시지는 AH 가 보호하는 서비스 집합을 나타내기 위해 컨트롤러에 의해 전송된다. 다음 예에서 서비스는 고정 IP 주소로 지정되며, 이는 AH 가 SDP 게이트웨이인 경우의 예이다. 대신 AH 가 해결해야 할 필요가 있는 호스트명으로 서비스를 지정하는 것도 가능하다. AH 가 서비스를 실행 중인 호스트의 일부인 경우, AH 가 액세스를 제어하는 서비스를 식별하는 데 IP 주소 대신 id 또는 name 필드를 사용할 수 있다.

|      |                                  |
|------|----------------------------------|
| 0x04 | JSON formatted array of Services |
|------|----------------------------------|

데이터 플레인의 JSON 규격은 다음과 같다.

| Format                                                                                                                                                                                                                                                                     | Example                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>{ "services":<br/>  [<br/>    { "port": &lt;Server port&gt;, "id":<br/>      &lt;256-bit Service ID&gt;, "address":<br/>      &lt;Server IP&gt;, "name": &lt;service<br/>      name&gt;<br/>      "type" : &lt;protocol type tag &gt;<br/>    },<br/>  ],<br/>}</pre> | <pre>{ "services":<br/>  [<br/>    { "port": "443",<br/>      "id": "123445678",<br/>      "address": "10.2.1.123",<br/>      "name": "Marketing Web App",<br/>      "type": "HTTPS"<br/>    },<br/>  ],<br/>}</pre> |



TCP 또는 UDP 기반 서비스 또는 다른 프로토콜을 사용하는 서비스(예, ICMP)를 설명하는 데 사용할 수 있다.

## h. 개인용 예약

아래 명령어(0xff)는 AH와 컨트롤러 사이에 비 표준 메시지를 송수신하기 위해 예약되어 있음을 의미한다.

|      |                |
|------|----------------|
| 0xff | User-specified |
|------|----------------|

## IH-컨트롤러 프로토콜

IH-컨트롤러 프로토콜은 네트워크 라우팅 및 패킷 배열을 조작하며, 구현 상세 정보는 전송 유형(예, 신뢰를 보증하는 TCP 또는 메시지를 보낸 후 회신을 기다리지 않는 UDP)에 의존한다.

### IH와 컨트롤러 간 시퀀스 다이어그램

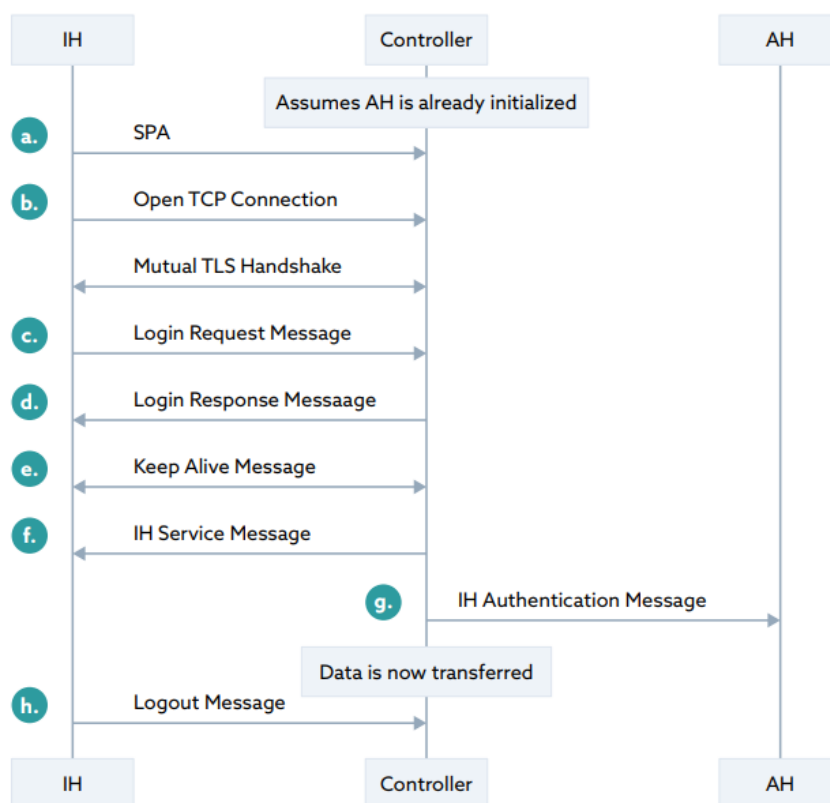


Figure 11: Initiating Host connects to the SDP Controller

다음 하위 절에서는 IH와 컨트롤러 사이에서 주고받게 되는 다양한 메시지와 그 형식을 정의한다. 기본적인 프로토콜은 다음과 같다.

|                 |                                                  |
|-----------------|--------------------------------------------------|
| Command (8-bit) | Command specific data of command-specific length |
|-----------------|--------------------------------------------------|

## a. SPA

SPA 패킷은 IH로부터 컨트롤러로 접속을 요청하기 위해 앞에서 설명한 포맷에 따라 송신된다.

## b. TCP 접속 및 상호 인증된 통신 설정

SPA 패킷을 전송한 후 IH는 컨트롤러에 대한 TCP 연결을 확립한다. 컨트롤러가 SPA 패킷이 유효하다고 판단할 경우 해당 TCP 연결을 허용한다. 이후 mTLS 연결 설정에 필요한 상호 인증이 뒤따른다.

UDP 기반 DTLS의 경우 UDP는 connectionless 형 프로토콜이므로 논리적인 접속이 된다.

## c. 로그인 (SDP Join) 요청 메시지

로그인 요청 메시지는 IH가 사용 가능하며, SDP에 포함되고 싶다는 것을 나타내기 위해 IH로부터 컨트롤러에 전송된다. IH의 로그인 요청 시 IH를 식별하고 컨트롤러에 대해서 인증하기 위한 증명서를 포함하는 것이 될 수 있도록 해야 한다. 또한 이 로그인 요청은 본 문서에서 설명한 on-boarding 플로우 후에 발생한다. 이 로그인 요청은 세션당 한 번(예, 사용자가 디바이스를 처음 켤 때 하루 한 번) 발생한다.

|      |                          |
|------|--------------------------|
| 0x00 | No command-specific data |
|------|--------------------------|

## d. 로그인 응답 메시지

로그인 응답 메시지는 로그인 요청이 성공했는지 여부를 나타내며, 성공했을 경우 IH 세션 ID를 제공하기 위해 컨트롤러에 전송된다. 컨트롤러가 IH의 로그인 요청을 거부할 수 있다. 예를 들어 자격 증명이 무효한 경우 등이다. 또는 컨트롤러가 시스템의 라이선스 또는 동시 접속 수의 제한을 적용하고 있어, 이러한 이유로 IH의 로그인 시도를 거부할 가능성도 있다.

|      |                       |                                                                                   |
|------|-----------------------|-----------------------------------------------------------------------------------|
| 0x01 | Status Code (16 bits) | IH Session ID (32 bits) and optionally, renewed SPA keys and TLS keys for the IH. |
|------|-----------------------|-----------------------------------------------------------------------------------|

| Format                                                                                                                                                                                                                                     | Example                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>{  "session_id" : &lt;256 bits&gt;,   "credentials":     ["spa_encryption_key":&lt;64 bit&gt;,        "spa_hmac_key": &lt;64 bit&gt;,        "tls_key": &lt;file contents&gt;,        "tls_cert" &lt;file contents&gt;      ] }</pre> | <pre>{  "session_id": "0x1234..."   "credentials":     ["spa_encryption_key":"aldskf...",        "spa_hmac_key": "asldjfk...",        "tls_key": "tls_key",        "tls_cert": "tls_cert"      ] }</pre> |

## e. Keep-Alive 메시지

Keep-Alive 메시지는 IH 또는 컨트롤러에 의해 전송되어, 현재 active 상태임을 나타낸다.

|      |                          |
|------|--------------------------|
| 0x03 | No command-specific data |
|------|--------------------------|

## f. IH 서비스 메시지

서비스 메시지는 사용 가능한 서비스 목록과 해당 서비스를 보호하는 AH의 IP 주소 또는 호스트 명을 IH에 제공하기 위해 컨트롤러를 통해 전송된다. 이 메시지는 IH가 서비스에 연결할 수 있도록 충분한 정보를 포함해야 한다. 호스트 명/IP 주소는 IH가 직접 연결할 수 있는 AH가 된다. 앞서 "배포 모델"에서 설명한 것과 같이 AH와 다른 호스트/IP로 동작하고 있을 수 있다.

서비스 ID는 IH가 AH와 통신할 때 대상이 되는 서비스를 식별하기 위해 사용한다.

|      |                                  |
|------|----------------------------------|
| 0x06 | JSON formatted array of Services |
|------|----------------------------------|

JSON 규격은 다음과 같다.

| Format                                                                                                                                                                                                                                 | Example                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>{ "services": [<br/>  { "address" : &lt;AH IP&gt;,<br/>    "id": &lt;256-bit Service ID&gt;, "name":<br/>    &lt;service name&gt;,<br/>    "type" : &lt;service type&gt;,<br/>    "port": &lt;Server port&gt;<br/>  }<br/>]</pre> | <pre>{ "services": [<br/>  { "address" : "10.2.1.34",<br/>    "id": "123445678",<br/>    "name": "FinanceApp", "type" :<br/>    "HTTPS",<br/>    "port" : "8443" }<br/>]</pre> |

## g. IH 인증 메시지

AH 역할은 보호된 리소스의 리스트에 대한 액세스를 허용하기 전에 인증 요청이 확인되는 것을 보증하는 것이다. IH 인증 메시지는 컨트롤러에 의해 AH 로 전송되어 새 IH 가 검증되었으며 AH 가 지정된 서비스에 대해 이 IH 에 액세스할 수 있도록 허용해야 함을 나타낸다.

이 메시지는 컨트롤러부터 AH 에 전송되지만 IH 가 컨트롤러에 인증할 때 시작된다.

|      |                                        |
|------|----------------------------------------|
| 0x05 | JSON formatted array of IH information |
|------|----------------------------------------|

JSON 규격은 다음과 같다.

| Format                                                                                                                                                                                                                                                                                                                               | Example                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>{ "IH Authenticators":<br/>  "IH": &lt;IH/Device Pair&gt;,<br/>  "session_id":<br/>  &lt;256-bit IH Session ID&gt;,<br/>  "hmac_seed": &lt;256-bit SPA HMAC seed<br/>  for the IH&gt;,<br/>  "hotp_seed": &lt;256-bit SPA HOTP seed<br/>  for the IH&gt;<br/>  "id": [ &lt;array of 256-bit service<br/>  IDs&gt; ]<br/>}</pre> | <pre>{ "IH Authenticators":<br/>  "IH": "IH/DeviceID",<br/>  "session_id": "4562",<br/>  "hmac_seed": "###",<br/>  "hotp_seed": "###",<br/>  "id": [ "123445678", "9012345"<br/>]</pre> |

컨트롤러로부터 AH 에 전송되는 이 페이로드의 정보는 AH 가 IH 로부터의 향후 연결을 검증하는 데 충분할 필요가 있는 것에 주의해야 한다. 위의 예에서는 AH 는 IH 고유의 디바이스 ID, 세션 ID, SPA 시드를 필요로 한다. 구현에 따라 다르게 접근할 수 있다.

## h. 로그아웃 요청 메시지

로그아웃 요청 메시지는 IH 가 SDP 세션을 종료하는 것을 나타내기 위해 IH 로부터 컨트롤러에 전송된다. 컨트롤러에서 보낸 응답 메시지는 송신되지 않으며 TLS 및 TCP 연결은 IH 또는 컨트롤러에 의해 종료되어야 한다. IH 는 on-board 상태로 유지되며 이후 새 세션을 다시 설정할 수 있다.

|      |                          |
|------|--------------------------|
| 0x07 | No command-specific data |
|------|--------------------------|

## z. 개인용 예약

아래 명령어(0xff)는 IH 와 컨트롤러 사이의 비표준 메시지 송수신을 위해 예약되어 있다. 그림 11 에서 해당 메시지는 표시되어 있지 않다.

|      |                |
|------|----------------|
| 0xff | User-specified |
|------|----------------|

## IH-AH 프로토콜

IH-AH 프로토콜은 네트워크 라우팅과 패킷 전송을 사용한다. 구현 세부 정보는 전송 유형(예, 신뢰를 보증하는 TCP 또는 메시지를 보낸 후 회신을 기다리지 않는 UDP)에 의존한다.

IH 가 유효한 SPA 패킷과 연결되고 상호 인증된 후에 한하여 AH 를 통해 서비스에 동적으로 연결이 설정된다. 통신 및 AH 는 IH 가 요청된 서비스에 액세스할 수 있는 권한이 있는지 확인한다. 그때까지 AH 에 의해 은폐된다.

## IH 와 AH 간 시퀀스 다이어그램

IH 와 AH 간 프로토콜의 시퀀스 다이어그램은 그림 12 와 같다.

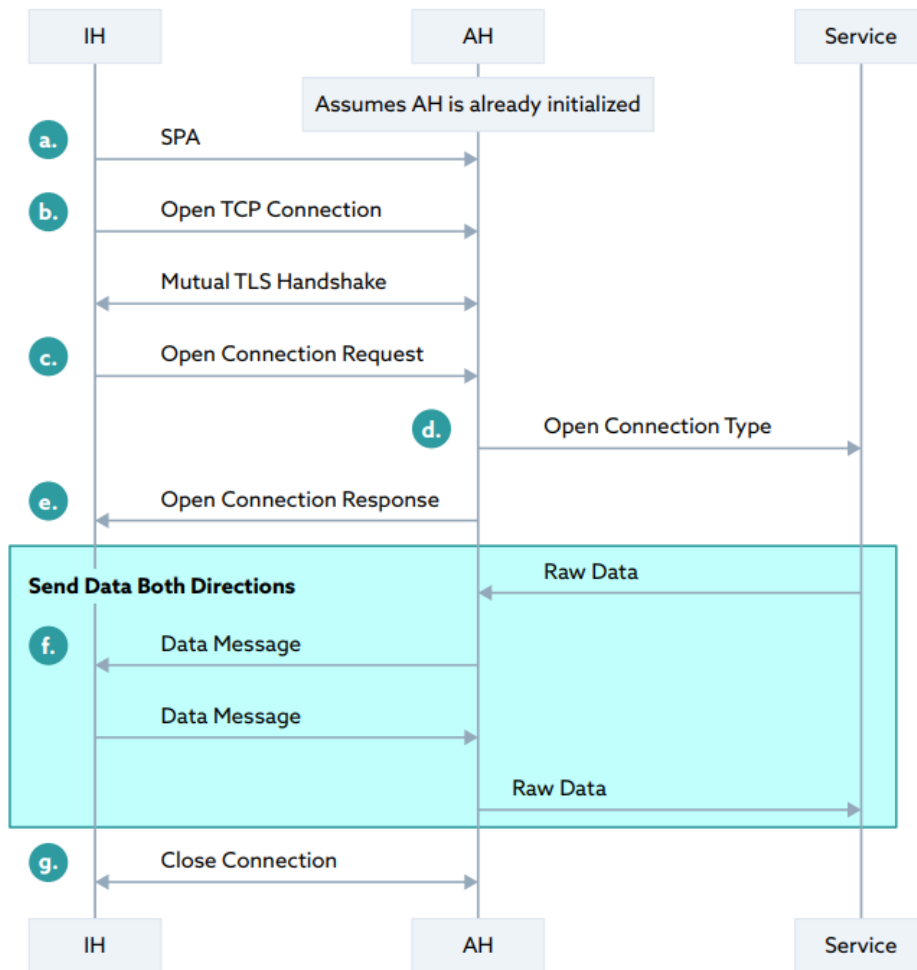


Figure 12: IH connects to an AH and then sends data to a Service

다음은 IH 와 AH 간 전달되는 다양한 메시지와 포맷을 정의한다. 기본 프로토콜은 다음과 같다.

|                 |                                                  |
|-----------------|--------------------------------------------------|
| Command (8-bit) | Command specific data of command-specific length |
|-----------------|--------------------------------------------------|

## a. SPA

SPA 패킷은 앞서 설명한 포맷에 따라 연결을 요청하기 위해 IH 에 의해 AH 로 전송된다.

## b. TCP 접속 및 상호 인증된 통신 설정

SPA 패킷을 송신 후 IH 는 AH 에 TCP 연결 설정을 시도한다. AH 가 SPA 패킷이 유효하다고 판단할 경우 TCP 연결이 설정되도록 허용한다. 이후 mTLS 연결 설정에 필요한 상호 인증이 뒤따른다.

UDP 기반 DTLS 의 경우, UDP 는 connectionless 이므로 논리적 연결이다.

### c. TCP 접속 요청 메시지

IH 는 특정 서비스에 대한 연결을 설정하겠다는 메시지를 AH 로 보낸다.

Service ID 는 컨트롤러가 각 원격 서비스에 대해 할당한 고유한 값이다. IH 는 Service ID 를 인식한다. Service ID 는 위에서 설명한 IH-컨트롤러 프로토콜의 IH 서비스 메시지 일부로 이전 컨트롤러에 의해 IH 로 전송되었기 때문이다.

Session ID 는 IH 와 AH 에서 특정 원격 서비스에 대한 서로 다른 TCP 연결을 구별하는 데 사용된다.

|      |                                                                    |
|------|--------------------------------------------------------------------|
| 0x07 | Service ID - 256-bit identifier<br>Session ID - 256-bit identifier |
|------|--------------------------------------------------------------------|

### d. 적절한 접속 유형 설정

이 단계는 구현과 SDP 구축 모델에 국한되며, AH 가 IH 를 대신하여 서비스에 대한 연결을 설정하는 데 사용된다. 단시간의 HTTPS 접속이나 초기 교환의 일부로 IH 에서 제공하는 애플리케이션 레벨의 인증이 필요한 연결(예, SSH)과 같은 특정 프로토콜에는 필요하지 않을 수 있다. SDP 대응 서비스는 애플리케이션 프로토콜 외부에서 AH 로부터 IH 또는 사용자의 컨텍스트를 전달하기 위해 해당 연결을 설정해야 할 수 있다.

### e. TCP 접속 응답 메시지

Open-response 메시지는 open 요청의 성공 여부를 나타내기 위해 AH 로부터 IH 에 전송된다. TCP 접속 요청 및 TCP 접속 응답 메시지는 IH 에 대해 비동기일 수 있으므로 Service ID 및 Session ID 코드는 IH 가 해당 반환 값을 해당 요청과 연결하는 데 유용하다.

|      |                       |                                                                    |
|------|-----------------------|--------------------------------------------------------------------|
| 0x08 | Status Code (16 bits) | Service ID - 256-bit identifier<br>Session ID - 256-bit identifier |
|------|-----------------------|--------------------------------------------------------------------|

### f. 데이터 메시지

데이터 메시지는 IH 또는 AH 에 의해 전송된다. Open connection 에 데이터를 푸시하기 위해 사용된다. 응답은 없다. 메시지는 구현에 따라 다르다. 일부 SDP 구현은 이와 같은 메시지로 애플리케이션 데이터를 패키징하는 반면, 다른 구현은 다른 접근 방식을 사용할 수 있다.

|      |                       |                                                                    |
|------|-----------------------|--------------------------------------------------------------------|
| 0x09 | Data Length (16 bits) | Service ID - 256-bit identifier<br>Session ID - 256-bit identifier |
|------|-----------------------|--------------------------------------------------------------------|

## g. 접속 종료 메시지

접속 종료 메시지는 IH 또는 AH 에 의해 전송된다. AH 에 의해 접속이 닫혔거나, IH 가 연결 종료를 요청하고 있음을 나타내는 데 사용된다. 응답은 없다.

|      |                                                                    |
|------|--------------------------------------------------------------------|
| 0x0A | Service ID - 256-bit identifier<br>Session ID - 256-bit identifier |
|------|--------------------------------------------------------------------|

## z. 사용자 정의 메시지

아래 명령어(0xff)는 IH 와 AH 간 모든 비표준 메시지에 대해 예약된다. 해당 메시지는 그림 12 에 나와 있지 않다.

|      |                |
|------|----------------|
| 0xff | User-specified |
|------|----------------|

# 로깅

서비스 가용성 및 성능, 서버의 보안을 결정하기 위한 로그를 생성하는 것은 모든 시스템과 제로 트러스트 구현의 요구사항이다.

## 로그 메시지의 필드

모든 로그는 다음 필드가 있다.

| 필드 명          | 의미                                                                                                                  |
|---------------|---------------------------------------------------------------------------------------------------------------------|
| time          | 로그 레코드가 생성된 시간                                                                                                      |
| name          | 이벤트에 대한 사람이 지정한 이름. 참고: 사용자 명, IP 주소, 호스트명 등 변형 가능한 데이터 nuggets 은 포함되지 않도록 해야 한다. 해당 정보는 로그 레코드의 추가 파일에 이미 포함되어 있다. |
| severity      | 이벤트 심각도는 debug 에서 critical 까지의 중요도(아래 참조)                                                                           |
| deviceAddress | 로그 레코드를 생성한 시스템의 IP 주소                                                                                              |

## 운영 로그

다음은 로깅해야 하는 운영 사용 사례 및 activity 목록이다.

signature\_id 는 이벤트 유형을 식별할 수 있는 식별자이다. 세번째 열에는 특정 메시지에 대해 로깅해야 하는 추가 필드가 있다.



| activity                                                         | signature_id                                       | data/ information to log                                                                                                                                                                 |
|------------------------------------------------------------------|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 컴포넌트 startup, shutdown, restart (예, 컨트롤러 기동, 호스트 재기동)            | ops:startup<br>ops:shutdown<br>ops:restart         | <b>reason</b> 은 재기동 또는 셧다운의 이유를 의미<br><br><b>component</b> 는 어느 컴포넌트가 영향을 받았는지를 의미                                                                                                       |
| 컴포넌트 간 접속 (컨트롤러, IH, AH, 3rd party 컴포넌트, DB) up, down, reconnect | ops:conn:up<br>ops:conn:down<br>ops:conn:reconnect | <b>src</b> 는 reporting entity 에 표시된 접속 출발지(IP 주소)<br><br><b>dst</b> 는 reporting entity 에 표시된 접속 목적지<br><br><b>reconnect_count</b> 는 재접속 시도한 횟수<br><br><b>reason</b> 은 통신이 단절된 이유를 알 수 있음 |

## 보안 / 접속 로그

보안 로그는 SDP 의 핵심이며, 대규모 인프라 공격을 탐지하기 위한 광범위한 맥락에서 중요하다. 따라서 이러한 로그는 SIEM(Security Information and Event Management)으로 전달할 때 매우 유용하다. 제로 트러스트 구현 시 필요하다.

signature\_id 는 식별자이며, 이벤트 종류를 식별할 수 있다. 세번째 열에는 특정 signature\_id 에 대해 기록해야 하는 추가 필드가 포함된다.

| activity  | signature_id      | data/ information to log                                                         |
|-----------|-------------------|----------------------------------------------------------------------------------|
| AH 로그인 성공 | sec:login         | <b>src</b> 는 컨트롤러에 표시되는 AH 의 IP 주소<br><br><b>AH Session ID</b> 는 AH 의 session ID |
| AH 로그인 실패 | sec:login_failure | <b>src</b> 는 컨트롤러에 표시되는 AH 의 IP 주소<br><br><b>AH Session ID</b> 는 AH 의 session ID |
| IH 로그인 성공 | sec:login         | <b>src</b> 는 컨트롤러에 표시된 IH 의 IP 주소<br><br><b>IH Session ID</b> 는 IH 의 session ID  |
| IH 로그인 실패 | sec:login_failure | <b>src</b> 는 컨트롤러에 표시된 IH 의 IP 주소<br><br><b>IH Session ID</b> 는 IH 의 session ID  |

|                        |                       |                                                                                                |
|------------------------|-----------------------|------------------------------------------------------------------------------------------------|
| SPA 인증                 | <i>sec:auth</i>       | <b>SPA</b> 패킷이 IH 에서 컨트롤러로 전송<br><b>SPA</b> 패킷이 AH 에서 컨트롤러로 전송<br><b>SPA</b> 패킷이 IH 에서 AH 로 전송 |
| 컴포넌트 인증(예, IH -> 컨트롤러) | <i>sec:connection</i> | <b>IH Session ID</b> 는 IH 의 session ID<br><b>AH Session ID</b> 는 AH 의 session ID               |
| 인바운드 연결 거부             | <i>sec:fw:denied</i>  | <b>src</b> 는 연결의 출발지<br><b>dst</b> 는 연결의 목적지                                                   |

다음은 완전한 정지 상태를 나타내는 간단한 시나리오를 나타내며, 어떤 로그 Entry 가 어디에 기록되는지 나타낸다. 이 시나리오에서는 컨트롤러가 다운되었다고 가정한다.

컨트롤러가 다운되다[로그 없음, 장애가 발생한 컴포넌트는 로깅되지 않는다].

1. IH 는 컨트롤러의 재접속을 n 회 시도한다  
*ops:conn:reconnect* 로그 메시지
2. N 회 시도 후, 클라이언트는 컨트롤러의 접속이 중단되었다고 선언하고 새 컨트롤러를 찾는다.  
*ops:conn:down* 로그 메시지(error 의 심각도를 포함)
3. IH 는 새로 찾은 컨트롤러에 접속한다  
*ops:conn:up* 로그 메시지
4. 사용 가능한 컨트롤러가 존재하지 않은 경우  
*ops:conn:down* 로그 메시지(critical 심각도를 포함)

유사한 예로, 경고 없이 클라이언트가 다운된 경우(예, 노트북을 닫은 경우)가 있다. 이 경우 컨트롤러는 AH 및 연결 장애를 감지한다. 각 오류 심각도와 함께 *ops:conn:down* 로그 메시지를 기록한다.

다음은 전체 사용자 로그인(AH 에 연결하는 IH)의 예이다.

5. IH 는 컨트롤러에 접속  
*sec:auth* 로그 메시지 – 컨트롤러에 대한 SPA 인증  
*ops:conn:up* 로그 메시지
6. IH 는 컨트롤러와 상호 인증한다  
*sec:connection* 로그 메시지
7. IH 가 AH 에 접속  
*sec:auth* 로그 메시지  
*ops:conn:up* 로그 메시지

# 요약

SDP 는 제로 트러스트 구현 시 유효하다.<sup>19</sup> 개정된 규격에 의해, 기업은 애플리케이션, 네트워크, 사용자, 디바이스, 데이터를 보호하기 위해 제로 트러스트의 패러다임을 채택하는 것을 권장한다. 이는 클라우드로의 전환과 위협의 격화를 고려할 때 매우 중요한 과제가 되고 있다.

SDP 는 기업의 IaaS(Infrastructure as a Service) 클라우드 서비스, 네트워크 기능 가상화, SDN(Software-Defined Networking) 및 IoT 애플리케이션을 안전하게 보호하는 것으로 입증되었다.

SDP 규격 버전 2.0 은 출시되기까지 오랜 기간이 걸렸다. 버전 1.0 은 약 8 년전인 2014 년 4 월에 배포되었다. 이 기간 동안 제로 트러스트 원칙에 대한 열의와 채용이 진행되어, 그에 상응하는 SDP 기반 솔루션에 대한 관심과 도입이 확대되어 왔다.

SDP 규격 버전 2.0 은 다음 영역을 확장 및 수정한다(명확화 및 확장을 기반).

- SDP 와 제로 트러스트의 관계
- SDP 아키텍처와 컴포넌트
- Onboarding 과 액세스 워크 플로우
- SPA 메시지 포맷, UDP 이용, 및 대체 수단
- IoT 디바이스 및 액세스 정책에 대한 초기 논의

추가적으로, 다음과 같은 세 가지 SDP 하위 프로토콜에서 향상된 시퀀스 다이어그램과 연결 및 메시지에 대한 설명을 제공했다.

- AH to Controller
- IH to Controller
- IH to AH

SDP WG 은 향후 연구 및 출판을 위해 고려하고 있는 주제 중 일부는 다음과 같다.

- 제로 트러스트 정책 모델 및 SDP(즉, Policy decision point (PDP) 및 해당 policy enforcement point (PEP)를 통한 액세스)이다.
- SDP 로 실현하는 IoT 의 제로 트러스트
- SDP 자격 증명을 안전하게 저장 및 교체
- 디바이스 유효성 검증

향후 협업 및 위 주제에 대한 출판을 기대한다.

<sup>19</sup> <https://www.helpnetsecurity.com/2020/05/29/sdp-zero-trust/>

# References

Cloud Security Alliance (CSA), Integrating SDP and DNS: Enhanced Zero Trust policy enforcement - Pending publication Q1 2022 available at <https://cloudsecurityalliance.org/artifacts/integrating-sdp-and-dns-enhanced-zero-trust-policy-enforcement/>

Cloud Security Alliance (CSA), Software Defined Perimeter (SDP) and Zero Trust, May 27, 2020, available @ <https://cloudsecurityalliance.org/artifacts/software-defined-perimeter-and-zero-trust/>

Cloud Security Alliance (CSA), Zero Trust Presentation to OMG (Object Management Group), SDP: The Most Advanced Zero Trust Architecture, available @ <https://cloudsecurityalliance.org/artifacts/sdp-the-most-advanced-zero-trust-architecture/>

Cloud Security Alliance (CSA), SDP Architecture Guide version 2.0, published May 2019, available @ <https://cloudsecurityalliance.org/artifacts/sdp-architecture-guide-v2/>

Cloud Security Alliance (CSA), SDP Glossary, published June 2018, available @ <https://downloads.cloudsecurityalliance.org/assets/research/sdp/SDP-glossary.pdf>

Cloud Security Alliance (CSA), SDP as a DDoS Defense Mechanism, published October 2019, available @ <https://cloudsecurityalliance.org/artifacts/software-defined-perimeter-as-a-ddos-prevention-mechanism/>

Waverley Labs, SDP Center, Open Source Reference Implementation (funded by DHS), available @ <http://sdpcenter.com/test-sdp/>

Cloud Security Alliance (CSA) Software-Defined Perimeter (SDP) Specification 1.0, published April 2014, available @ <https://cloudsecurityalliance.org/artifacts/sdp-specification-v1-0/>

Zero Trust Security: An Enterprise Guide, by Jason Garbis and Jerry W. Chapman, Apress, 2021, available @ <https://www.apress.com/us/book/9781484267011>

Zero Trust Networks: Building Secure Systems in Untrusted Networks, by Evan Gilman and Doug Barth, O'Reilly, 2017, available @ <https://www.oreilly.com/library/view/zero-trust-networks/9781491962183/>

National Institute of Standards and Technology (NIST), NIST SP 800-207, Zero Trust Architecture document, August 2020, available @ <https://csrc.nist.gov/publications/detail/sp/800-207/final>

Cybersecurity and Infrastructure Security Agency Cybersecurity (CISA) Division, Zero Trust Maturity Model (Draft), June 2021 Version 1.0 (Comment Period Ends October 1, 2021), available @ <https://www.cisa.gov/publication/zero-trust-maturity-model>

Office of Management and Budget, Federal Zero Trust Strategy (Draft), (Comment Period Ends September 21, 2021, available @ <https://zerotrust.cyber.gov/federal-zero-trust-strategy/>

Cybersecurity and Infrastructure Security Agency (CISA), in collaboration with the United States Digital Service and FedRAMP, (Comment Period Ends October 1, 2021), available @ <https://zerotrust.cyber.gov/cloud-security-technical-reference-architecture/>

Executive Order 14028 on Improving the Nation's Cybersecurity, May 12, 2021, available @ <https://www.whitehouse.gov/briefing-room/presidential-actions/2021/05/12/executive-order-on-improving-the-nations-cybersecurity/>

National Security Agency (NSA), Embracing a Zero Trust Security Model, February 2021, available @ <https://www.nsa.gov/Press-Room/Cybersecurity-Advisories-Guidance/smdpage11747/2/>

*Adoption of the Software-Defined Perimeter (SDP) Architecture for Infrastructure as a Service*, by J. Singh, A. Refaey and J. Koilpillai, in Canadian Journal of Electrical and Computer Engineering, vol. 43, no. 4, pp. 357-363, Fall 2020, doi: 10.1109/CJECE.2020.3005316. <https://ieeexplore.ieee.org/abstract/document/9240082>

*On IoT applications: a proposed SDP framework for MQTT*, Electronics Letters, by Refaey, A.; Sallam, A.; Shami, A.: 2019, 55, (22), p. 1201-1203, DOI: 10.1049/el.2019.2334 IET Digital Library, [https:// digital-library.theiet.org/content/journals/10.1049/el.2019.2334](https://digital-library.theiet.org/content/journals/10.1049/el.2019.2334)

P. Kumar, Abdallah Moubayed, Ahmed Refaey, Abdallah Shami, and Juanita Koilpillai "Performance Analysis of SDP for Secure Internal Enterprises," IEEE Wireless Communications and Networking Conference (WCNC), 1-6, 2019.

J. Singh, A. Refaey and A. Shami, "Multilevel Security Framework for NFV Based on Software Defined Perimeter," in IEEE Network, vol. 34, no. 5, pp. 114-119, September/October 2020, doi: 10.1109/MNET.011.1900563.

Ahmed Sallam, Ahmed Refaey, and Abdallah Shami, "On the Security of SDN: A Completed Secure and Scalable Framework Using the Software-Defined Perimeter," IEEE Access, Accepted, August 2019. (Impact factor: 4.098).

# 부록 A: SDP, SDN & NFV

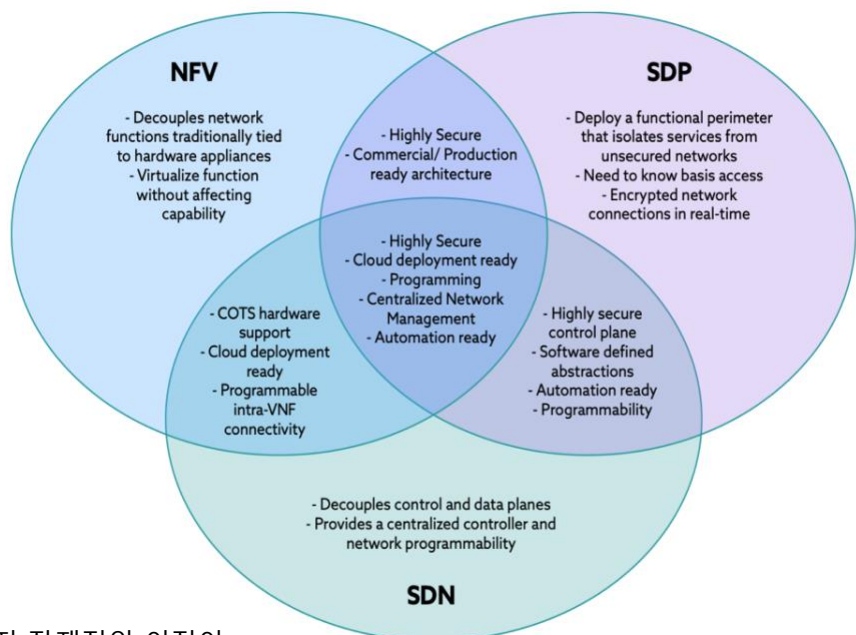
클라우드 컴퓨팅 환경에서 SDN(Software Defined Network)과 NFV(Network Function Virtualization) 기술은 IH 환경(frontend)과 AH 환경(Backend)의 양 쪽 과제<sup>20</sup>에 효율적으로 대응하고, 온디맨드, 종량제, 서비스로서 리소스 제공의 이점을 제공한다. SDN 과 NFV 는 리소스(무선 및 컴퓨팅 리소스)의 활용도를 높이기 위해 이기종 네트워크 라우팅을 채택하고 조정할 수 있는 기반을 마련한다. IH 환경 과제는 무선 모바일 네트워크 또는 edge 네트워크의 액세스 계층의 통신 측면의 보안과 관련된 반면, AH 환경 과제는 라우팅, 스위칭, 보안, accounting 및 과금과 같은 네트워크 기능에 관한 것이다.

NFV 의 주요 과제 중 하나는 리소스 고갈이 있다. 특정 물리 서버의 리소스를 집중적으로 사용하는 소프트웨어가 해당 리소스를 모두 사용하여 VM 의 가용성에 영향을 줄 가능성이 있다. 이러한 상황은 물리적 서버의 공유 환경이 리소스 경합의 심각성을 확대하기 때문에 발생하며, 특히 여러 VM 에서 동일한 리소스 집약형 소프트웨어를 동시에 실행하는 경우 리소스 경쟁의 심각함을 증대한다. 이 문제는 SDP 를 사용하는 것으로 해결할 수 있다. SDP 컨트롤러를 통해 DoS 와 마찬가지로 리소스 소진으로 인해 조정되는 VM 을 감지하고 동적으로 문제를 해결하는 표준 운영 절차를 정의 및 구현하도록 함으로써 해결할 수 있다. NFV 의 또 다른 일반적인 위험은 셀프 서비스 포털 또는 클라우드 관리 콘솔을 통한 계정 또는 서비스의 탈취이다. 포털이나 콘솔에 대한 액세스가 End User 에 부여된 과도한 관리 권한을 통해 위험에 노출될 가능성을 높인다. 이 경우, SDP 는 이러한 위험을 제거하고 사용자의 역할과 기능에 따라 선택적으로 관리 제어를 사용하는 데 중요한 역할을 한다.

다음은 소프트웨어 정의의 광범위한 채택이 있다.

클라우드 서비스 공급자가 네트워크 (SDN)를 통해 네트워크 관리를 단순화한다. SDN 의 주요 과제는 적절한 인증, 액세스 제어, 개인 정보보호 및 네트워크 라우팅의 API 기반 오케스트레이션을 위해 데이터 무결성을 제공 여부이다. 여기서 SDP 는 SDN 지원 네트워크 인프라의 객체 간 연결 및 네트워크 액세스를 제한하는 연결의 조정 기능을 제공할 수 있다.

SDP 와 SDN 간 통합으로 인한 몇 가지 잠재적인 이점이 있다. 특히 확장 가능하며 관리되는 보안 솔루션을 제공한다.



<sup>20</sup> J. Singh, A. Refaey and J. Koilpillai, "Adoption of the Software-Defined Perimeter (SDP) Architecture for Infrastructure as a Service," in Canadian Journal of Electrical and Computer Engineering, vol. 43, no. 4, pp. 357-363, Fall 2020, doi: 10.1109/ CJECE.2020.3005316.

SDN 과 NFV 는 네트워크 가상화 삼각형의 두 측면, SDP 는 세번째 측면으로 간주한다. SDN 과 SDP 모두 네트워킹 분야에서 운영되며 이름은 유사하나, SDN 은 네트워크 운영을 조정하는 두뇌로 간주될 수 있으며, SDP 는 큰 방해 없이 제로 트러스트 개념으로 안정적인 네트워크 연결을 제공한다.

## 부록 B: OSI / SDP 컴포넌트 매핑

이 섹션에서는 OSI 네트워크 계층을 클라우드 계층, 클라우드 스택 기능 및 SDP 컴포넌트에 매핑하는 기능을 제공한다. 이 매핑은 SDP 가 네트워킹 및 클라우드 모델의 여러 계층에서 보안을 제공하는 데 논리적으로 어떻게 도움이 되는지 보여주는 것을 목적으로 한다.

| OSI Layer    | Cloud Layer                  | Cloud Stack                                   | SDP Component                            |
|--------------|------------------------------|-----------------------------------------------|------------------------------------------|
| Application  | Application                  | End-User 계층 – 애플리케이션 및 비즈니스 가치 제공             | SDP 클라이언트 – 애플리케이션에 대한 사용자 액세스 제공        |
| Presentation | Service                      | Middleware – 애플리케이션이 계층에서 사용하는 컴포넌트           | SDP 컨트롤러 – 리소스에 대한 사용자 액세스를 중개하는 미들웨어 역할 |
| Session      | Image                        | OS – 기반이 되는 가상화를 적절하게 관리                      | SDP 정책 – 방화벽 규칙 및 세션 관리 적용               |
| Transport    | Software-Defined Data Center | Cloud API – resourceful/사용자에게 연결된 가상 자산 생성 가능 | 상호 TLS                                   |
| Network      | Hypervisor                   | 가상화 – 컴퓨팅, 스토리지 및 모니터링 가상화 제공                 | SDP Accepting Hosts (게이트웨이)              |
| Data Link    | Infrastructure               | 하드웨어 – 데이터 센터 내 물리 디바이스                       | n/a                                      |
| Physical     | Infrastructure               | 하드웨어 – 데이터 센터 내 물리 디바이스                       | n/a                                      |